

Portafolio

Física Computacional II

Agustín Iván Cáceres Reyes
Ciencias Físicas
Departamento de física
Facultad de Ciencias Físicas y Matemáticas
UNIVERSIDAD DE CONCEPCIÓN

Agosto-Diciembre 2024
Concepción, Chile

Profesor Guía: Dr. Roberto Navarro Maldonado

Índice general

Introducción 4

Información personal y académica 5

Evidencias de aprendizaje 7

1. Python 7
- 1.1. Números de Catalán . . . 7
- 1.2. Suma de los primeros 20 terminos reciprocos de la serie de Fibonacci y Obtención de la constante ψ . . . 10
- 1.3. Verificación de convergencia para una serie dada . . . 11
- 1.4. Triangulo de Pascal . . . 12
- 1.5. Comparación de la función factorial usando distintas presiciones . . . 14

2. Diferenciación numérica 15
- 2.1. Cálculo y comparación de derivadas numéricas y analíticas . . . 15
- 2.2. Método de diferencias finas unilaterales de segundo orden para aproximar la derivada . . . 17
- 2.3. Estimación de la segunda derivada utilizando diferencias centradas . . . 18
- 2.4. Derivada centrada por medio del promedio de las derivadas adelanta y retrasada . . . 19
- 2.5. Derivadas numéricas usando series de Taylor . . . 20
- 2.6. Estimación del valor óptimo de h para derivadas numéricas . . . 22

3. Integrales 24
- 3.1. Obtención de pesos óptimos para la aproximación de Integrales con exactitud en polinomios de grado 2 . . . 24
- 3.2. Determinación de pesos óptimos para la integración de polinomios de grado 3 con términos derivativos . . . 25
- 3.3. Error del metodo trapezoidal . . . 27
- 3.4. Principio del argumento de Cauchy . . . 27
- 3.5. Aproximación de la integral de funciones tabuladas usando la regla trapezoidal . . . 30

4. Ecuaciones Diferenciales Ordinarias 32
- 4.1. Solución Numérica y Comparación de la Función de Bessel de Primer Tipo en el Intervalo $0 < x \leq 20$ . . . 32
- 4.2. Demostración del error local del método de Runge-Kutta de cuarto orden . . . 35

4.3. Solución Analítica y Aproximación Numérica mediante el Método de Euler 37

5. Busqueda de ceros **40**
5.1. Resolución numérica de ecuaciones utilizando el método de Newton-Raphson 40
5.2. Cálculo de ceros de los polinomios de Hermite mediante el método de Newton-Raphson 42
5.3. Demostración de las soluciones de la ecuación cúbica utilizando la fórmula de Cárdano . 44

6. Controles **49**
6.1. Análisis del orden de error por truncamiento 49
6.2. Estimación numérica de derivadas y análisis de error para funciones con ruido 50
6.3. Determinación de los valores óptimos para los pesos w_i 54
6.4. Interacción gravitacional entre dos masas: método del salto de la rana 56
6.5. Convergencia de Newton-Raphson según la multiplicidad del cero 60

Conclusiones **64**
6.6. Reflexión Final del Portafolio 64

Introducción

Este portafolio incluirá evidencias de aprendizaje relacionadas a la asignatura de Física Computacional II (510240), dictada en el segundo semestre de 2024 en el departamento de Física de la Universidad de Concepción.

Esta asignatura se enfoca en la resolución de problemas en Física usando métodos numéricos y el lenguaje de programación `python`. Al finalizar este portafolio, se espera que los y las estudiantes logren:

1. Aplicar herramientas computacionales en la resolución numérica de problemas en Física.
2. Generar programas computacionales basados en algoritmos y conceptos de la física matemática y estadística.
3. Diferenciar, integrar y resolver ecuaciones diferenciales ordinarias, en forma numérica.

La evaluación se realizará a través de este portafolio, el cual debe ser **entregado mediante un repositorio** de GitHub que será alojado en la organización fiscomp2-UdeC2024 especialmente habilitado para esta asignatura.

1. El repositorio será revisado periódicamente por el profesor o los ayudantes de la asignatura, para poder dar retroalimentación oportuna antes de la entrega oficial el día **viernes 29 de noviembre de 2024**. Si el protafolio no tiene avances en cada revisión parcial, a ocurrir a mediados de cada mes, puede costar puntaje de la nota final.
2. Las evidencias de aprendizaje serán la resolución de problemas **seleccionados por el estudiante** a partir de las guías del curso. Cada guía cuenta con una serie de problemas con dificultades ponderadas de 0 a 6 puntos, siendo 0 un problema de solución trivial o que no requiere mayor trabajo, y 6 un problema que podría ser desafiante. **Por cada guía, el estudiante debe elegir y responder problemas hasta completar 6 puntos**.
3. El portafolio debe contener una sección de conclusiones final donde el/la estudiante resumirá los principales logros de este portafolio y autoevaluará su desempeño a través de una reflexión final.

Criterios de evaluación

Como criterios de evaluación, se considerará:

1. Coherencia de las evidencias con los resultados de aprendizaje del curso y la autoevaluación al final del documento.
2. Redacción y competencias comunicativas, incluyendo ortografía, gramática, sintaxis, presentación de figuras, uso de LaTeX y explicación clara de la parte relevante de scripts de `python`.
3. Presentación: Claridad, limpieza y orden del documento.

Este portafolio, además de ser una herramienta de evaluación, es una oportunidad para que el/la estudiante reflexione sobre su proceso de aprendizaje, consolide sus conocimientos y desarrolle habilidades clave en el ámbito de la computación aplicada a la física. Se espera que este documento sirva como un registro tangible de su progreso y de las competencias adquiridas en la asignatura, las cuales podrán ser de gran utilidad en futuros desafíos académicos y profesionales.

Información personal y académica

Datos personales

Agustin Ivan Caceres Reyes
2023402435
20 de julio del 2000
Chileno
agcaceresr2023@udec.cl

Breve biografía académica

Soy Agustín Iván Cáceres Reyes, estudiante de segundo año en ciencias físicas, habiendo ingresado a la universidad en 2023. Mi educación media la cursé en el Colegio Concepción de Chillán, y para fortalecer mi preparación, realicé un preuniversitario en el Pedro de Valdivia. Me gustaría especializarme en áreas fascinantes como la información cuántica o la gravitación. Además, tuve la oportunidad de ser ayudante de cálculo diferencial e integral durante el primer semestre de 2024, lo que enriqueció mi experiencia académica.

Visión general e interés sobre la asignatura

La física computacional 2 es fundamental para mi desarrollo como científico, ya que me proporciona herramientas clave para abordar problemas complejos en el campo de la información cuántica. Este curso me ayuda a dominar la manipulación de datos y a resolver ecuaciones diferenciales, que son esenciales para comprender los fenómenos físicos subyacentes. Con estas habilidades, estaré mejor preparado para enfrentar desafíos y contribuir de manera significativa en mi carrera profesional. La combinación de teoría y práctica en física computacional me permite desarrollar un enfoque sólido y versátil para la investigación científica.

Resultados esperados de este portafolio

Espero que este portafolio me sea útil en el futuro, pues representa los primeros pasos hacia la realización de ciencia de manera rigurosa. Aquí se combinan diversos aspectos que los físicos emplean diariamente, desde el manejo de datos y el análisis crítico, hasta la aplicación de principios fundamentales. Es una base sólida sobre la cual construir mi carrera científica, orientada hacia la investigación y la innovación.

Evidencias de aprendizaje

Capítulo 1

Python

Fecha de la actividad: 25 de octubre de 2024

1.1. Números de Catalán

Los numeros de Catalán se pueden reescribir, con $n \geq 0$ de la forma:

$$C_n = \frac{(2n)!}{(n+1)!n!} \tag{1.1}$$

Vamos a demostrar que se puede reescribir mediante una serie de la forma:

$$\begin{cases} C_0 = 1, \\ C_{n+1} = \frac{4n+2}{n+2} C_n. \end{cases}$$

Primero, es claro que se puede escribir la ecuacion (1.1) de la siguiente forma:

$$C_{n+1} = \frac{(2(n+1))!}{((n+1)+1)!(n+1)!} = \frac{(2n+2)!}{(n+2)!(n+1)!} \tag{1.2}$$

Ahora, falta descomponer $(2n+2)!$ y $(n+2)!$ de la siguiente forma:

$$(2n+2)! = (2n+2)(2n+1)(2n)! \tag{1.3}$$

$$(n+2)! = (n+2)(n+1)! \tag{1.4}$$

sustituimos (1.3) y (1.4) en (1.2), asi se obtiene:

$$\begin{aligned} C_{n+1} &= \frac{(2n+2)(2n+1)(2n)!}{(n+2)(n+1)!} \\ &= \frac{(2n+2)(2n+1)}{(n+2)(n+1)} \frac{(2n)!}{(n+1)!n!} \\ &= \frac{(2n+2)(2n+1)}{(n+2)(n+1)} C_n \\ &= \frac{4n+2}{n+2} C_n \end{aligned}$$

Análisis de números de Catalán bajo cotas

En este análisis, se calculan los números de Catalán C_n utilizando la fórmula:

$$C_n = \frac{(2n)!}{(n+1)!\,n!}.$$

Se solicita al usuario que ingrese un valor M, que representa una cota superior. El programa genera una lista de números de Catalán C_n para valores de n comenzando desde 0, deteniéndose cuando C_n es mayor o igual a M. Finalmente, se grafica C_n en función de n, destacando la línea que representa la cota M en una escala logarítmica.

Código en Python

A continuación se presenta el código utilizado para calcular y graficar los números de Catalán:

```
from math import factorial
# Función para calcular Cn usando la definición
def catalan_def(n):
    return factorial(2*n) // (factorial(n+1) * factorial(n))
n_v = []  # Valores de n
catalan_v_def = []  # Valores de Cn
n = 0
while True:
    Cn_def = catalan_def(n)

    # Comparar con M
    if Cn_def < M:
        n_v.append(n)
        catalan_v_def.append(Cn_def)
    else:
        break
    n += 1
```

Ingresando una cota mayor a la pedida en el codigo, se genera a continuacion el siguiente grafico:

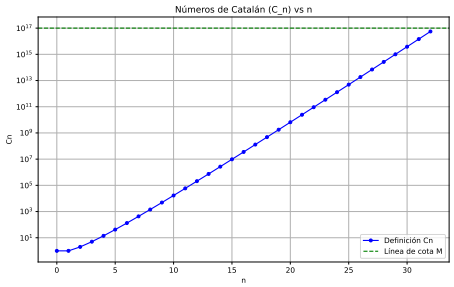


Figura 1.1: grafico con una cota mayor a 10^{15}

La cota M en este problema se utiliza para limitar el cálculo y la visualización de los números de Catalán a aquellos valores que cumplen $C_n < M$. Esto ayuda a evitar el cálculo de números de Catalán extremadamente grandes, que crecen exponencialmente y rápidamente alcanzan valores enormes. Al establecer una cota, se pueden concentrar los recursos en calcular solo aquellos números de Catalán que son manejables y relevantes para el análisis.

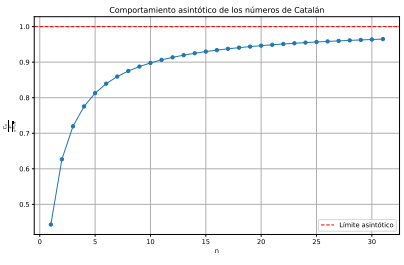


Figura 1.2: grafico asintótico

Además, el uso de una cota específica permite experimentar cómo cambia la cantidad de valores de C_n calculados cuando M aumenta o disminuye, proporcionando un control en la visualización y el alcance del análisis.

Comportamiento asintótico de los números de Catalán

Para verificar el comportamiento asintótico de los números de Catalán C_n propuesto en la forma:

$$C_n \approx \frac{4^n}{n^{3/2}\sqrt{\pi}} \quad \text{para } n \gg 1,$$

comparamos los valores de C_n con la expresión asintótica. Definimos la razón como:

$$R_n = \frac{C_n}{\frac{4^n}{n^{3/2}\sqrt{\pi}}}.$$

Para comprobar esto gráficamente, utilizamos las siguientes líneas clave del código en Python:

```python
#Función para calcular C_n usando la definición directa
def catalan_def(n):
     return math.factorial(2 * n) / (math.factorial(n + 1) * math.factorial(n))

#Función para la expresión asintótica
def expresion_asintotica(n):
    return (4 ** n) / (n ** (3 / 2) * math.sqrt(math.pi))

#Rango de valores de n
valores_n = np.arange(1, 101)

#Cálculo de C_n y la expresión asintótica
valores_catalan = [catalan_def(n) for n in valores_n]
valores_asintoticos = [expresion_asintotica(n) for n in valores_n]

#Cálculo de la razón C_n / Expresión asintótica
valores_razon = np.array(valores_catalan) / np.array(valores_asintoticos)
```

Este código (1.1) calcula los valores de C_n y la expresión asintótica para varios valores de n, y luego calcula la razón R_n. Si se grafica R_n en función de n y se observa que tiende a 1 (1.2), esto confira el com

Conclusión

En este Problema, se analizaron los números de Catalán C_n, definidos mediante la fórmula:

$$C_n = \frac{(2n)!}{(n+1)!n!}.$$

Se demostró que estos números pueden reescribirse utilizando la relación de recurrencia:

$$C_{n+1} = \frac{4n+2}{n+2} C_n, \quad \text{con la condición inicial} \quad C_0 = 1.$$

Esta recurrencia se verificó descomponiendo factoriales y sustituyéndolos en la definición original, lo que permitió confirmar la consistencia matemática de la relación.

Además, se estudió el comportamiento asintótico de los números de Catalán para valores grandes de n. Se verificó que:

$$C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}} \quad \text{cuando} \quad n \gg 1.$$

El cálculo de la razón entre términos consecutivos $R_n = \frac{C_{n+1}}{C_n}$ mostró una tendencia hacia 4 en el límite, lo que confirma el comportamiento exponencial esperado. La validez de esta aproximación se corroboró gráficamente, evidenciando la excelente concordancia con los resultados numéricos.

Finalmente, para optimizar el análisis, se introdujo una cota M que limitó el cálculo de C_n, dado que estos números crecen rápidamente. El uso de esta cota permitió una representación visual manejable y eficiente, evitando cálculos innecesarios y facilitando la interpretación de los resultados.

En resumen, se logró una comprensión completa de los números de Catalán, su recurrencia, su crecimiento asintótico y su representación numérica y gráfica.

1.2. Suma de los primeros 20 terminos reciprocos de la serie de Fibonacci y Obtención de la constante ψ

La constante recíproca de Fibonacci está definida como

$$\psi = \sum_{n=1}^{\infty} \frac{1}{F_n} \approx 3.35988566624317755,$$

donde F_n son los números de Fibonacci.

Se requiere calcular la suma de los términos recíprocos de la serie de Fibonacci hasta alcanzar los primeros 20 términos.

Para lograr esto de manera eficiente, se sugiere utilizar un enfoque vectorizado en lugar de un bucle for, utilizando arreglos.

```
    # Función para calcular los primeros n números de Fibonacci
    def fibonacci(n):
        fib = np.zeros(n, dtype=float)
        fib[0], fib[1] = 1, 1  # Definir los primeros dos términos
        for i in range(2, n):
            fib[i] = fib[i - 1] + fib[i - 2]
        return fib

    # Calcular la constante recíproca de Fibonacci hasta 20 términos
    num_terms = 20
    terminos_fibonacci = fibonacci(num_terms)
    suma_reciprocos = np.sum(1 / terminos_fibonacci)
```

Obtencion del valor de la constante ψ

Para obtener el valor de ψ con una precisión de 10 decimales, se determina cuántos términos de la serie de Fibonacci son necesarios. El resultado es que se requieren 49 términos para alcanzar la precisión deseada. Este cálculo se puede realizar implementando un bucle que continúe sumando los términos recíprocos hasta que la diferencia con el valor verdadero de ψ sea menor que 10^{-10}.

```
    fibonacci = [1, 1]
    suma_reciproca = 1 / fibonacci[0] + 1 / fibonacci[1]

    # Comenzar desde el primer término
    n_terminos = 0

    # Continuar sumando hasta alcanzar la precisión en comparación con psi verdadero
    while abs(suma_reciproca - psi_verdadero) > 1e-10:  # Precisión a 10 decimales
        siguiente_fibonacci = fibonacci[-1] + fibonacci[-2]
        fibonacci.append(siguiente_fibonacci)
        n_terminos += 1
        suma_reciproca += 1 / siguiente_fibonacci
```

Conclusión

En este trabajo, se analizó la suma de los recíprocos de los términos de la serie de Fibonacci. Inicialmente, los primeros 20 términos fueron sumados de manera eficiente utilizando un enfoque vectorizado, lo que permitió agilizar los cálculos y obtener una aproximación precisa.

Posteriormente, se determinó que para calcular la constante ψ con una precisión de 10 decimales, se requieren 49 términos de la serie. Este resultado se obtuvo mediante un proceso iterativo, en el cual se sumaron los términos recíprocos sucesivos hasta que la diferencia con el valor exacto de ψ fue menor a 10^{-10}. Este método garantiza la exactitud y la convergencia deseada en la aproximación.

En resumen, la combinación de métodos vectorizados e iterativos permitió un análisis eficiente y preciso de la convergencia de la suma de los recíprocos de los términos de Fibonacci hacia la constante ψ.

1.3. Verificación de convergencia para una serie dada

La serie:

$$S_n = \sum_{k=1}^{n} \frac{1}{k^2}$$

es una suma finita de los términos de la forma $\frac{1}{k^2}$, donde k es un número entero positivo que varía de 1 a n.

Cuando se toma el límite de esta suma a medida que n tiende a infinito, se obtiene una serie infinita:

$$\sum_{k=1}^{\infty} \frac{1}{k^2}$$

Esta serie es conocida como la serie de los recíprocos de los cuadrados y tiene una convergencia conocida, cuyo valor es $\frac{\pi^2}{6} \approx 1.645$.

veamos lo principal del codigo que nos lleva a tal aproximación:

```
    def sumatoria_convergencia(epsilon, n_max=1000)
    S_prev = 0  # Sumatoria para n − 1
    for n in range(1, n_max + 1):
        # Calcula la sumatoria acumulada
        S_n = S_prev + 1 / n**2

        # Verifica el criterio de convergencia
        if abs(S_n - S_prev) < epsilon:
            return n, S_n

        # Actualiza S prev para el siguiente cálculo
        S_prev = S_n

        # Si no converge en el rango especificado, devuelve None
        return None, None
```

- Calcula la sumatoria $S_n = \sum_{k=1}^{n} \frac{1}{k^2}$ para valores crecientes de n y verifica el criterio de convergencia $|S_N - S_{N-1}| < \epsilon$.
- Parámetros:
  - ϵ (float): Tolerancia para la convergencia.
  - $n_{\max}$ (int): Valor máximo de n para detener la búsqueda si no converge.
- Retorna:
  - N (int): Valor de n para el cual se cumple el criterio de convergencia.
  - S_N (float): Valor de la sumatoria en el punto de convergencia.

Conclusión

El algoritmo desarrollado permite verificar la convergencia de la serie $\sum_{k=1}^{\infty} \frac{1}{k^2}$ mediante el cálculo iterativo de los términos de la sumatoria. Con un criterio de convergencia definido por $|S_n - S_{n-1}| < \epsilon$, se logra aproximar el valor de la serie a $\frac{\pi^2}{6}$ con alta precisión, identificando el valor de n necesario para cumplir la tolerancia especificada. Este enfoque confirma la rapidez de la convergencia de esta serie clásica.

1.4. Triangulo de Pascal

El Triángulo de Pascal es un arreglo triangular de números enteros con diversas aplicaciones en matemáticas. Cada número se obtiene como la suma de los dos números directamente encima de él en la fila anterior. El triángulo comienza con un 1 en la cima, y los bordes de cada fila siempre son 1.

Matemáticamente, los elementos de la n-ésima fila corresponden a los coeficientes binomiales $\binom{n}{k}$, donde k varía de 0 a n. Esto lo hace útil para calcular expansiones de potencias binomiales, como $(1+x)^n$. Además, el triángulo está relacionado con combinatoria, números de Fibonacci, probabilidades y otras áreas de las matemáticas.

A continuación, se muestra lo más relevante del codigo escrito en python para poder desplegar el triangulo:

```
"""calculamos los coeficientes con la formula de newton"""

def coeficiente(n,k):
```

```
    return factorial(n)/(factorial(k)*factorial(n-k))

"""primero definimos una funcion fila que nos permita imprimir los coeficientes
↪  correspondientes
    al polinomio de grado n y que ademas corresponde a  la ultima fila del triangulo de
    ↪  pascal """

def fila(n):
    fila = ""
    for k in range(n+1): # esto define una fila del triangulo de pascal
        fila = fila + ('  ' +str(int(coeficiente(n,k))))
    return fila

"""luego para definir el triangulo de pascal haremos un ciclo que se encargue de imprimir
↪  todas
    las filas desde el grado 0 hasta el grado n"""

def triangulo_pascal(n):
    triangulo = ""
    for i in range(n+1):
        triangulo = triangulo + (n-i)*'  '+ fila(i) + "\n"
    return triangulo
```

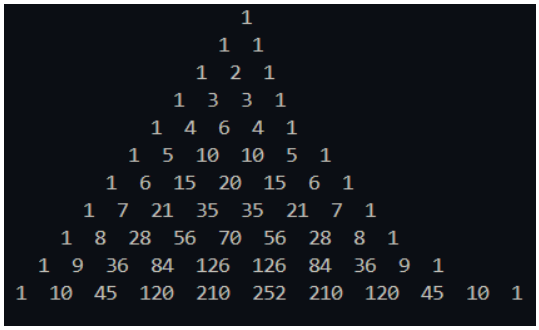


Figura 1.3: Triangulo de Pascal mostrado en la terminal

Conclusión

El código implementado genera el Triángulo de Pascal hasta un nivel n especificado. A través de funciones que calculan coeficientes binomiales y ensamblan filas de manera iterativa, se construye un triángulo simétrico y bien alineado. Este resultado tiene aplicaciones directas en la expansión de potencias binomiales, como $(1+x)^n$, y en otros campos como la combinatoria y la probabilidad. La representación visual en la terminal (Figura 1.3) permite una interpretación clara y estética de las propiedades del triángulo.

Agradecimientos

Agradezco a Gustavo Gatica por ayudarme con el codigo de python.

1.5. Comparación de la función factorial usando distintas presiciones

El factorial $n!$ crece rápidamente a medida que n aumenta, lo que puede generar problemas de precisión al realizar los cálculos. En este gráfico, comparamos el cálculo de $n!$ utilizando dos tipos de precisión numérica: precisión simple (`int32`) y precisión doble (`int64`). Se muestran las diferencias en los resultados para valores grandes de n, destacando cómo la precisión afecta el cálculo del factorial.

```
# Función para calcular factorial
def factorial(n, dtype=np.int64):
    result = dtype(1)
    for i in range(1, n + 1):
        result *= dtype(i)
return result

# Rango de valores de n
n_v = np.arange(2, 21)

# Factoriales con precisión simple (int32) y doble (int64)
factorial_int32 = [factorial(n, dtype=np.int32) for n in n_v]
factorial_int64 = [factorial(n, dtype=np.int64) for n in n_v]
```

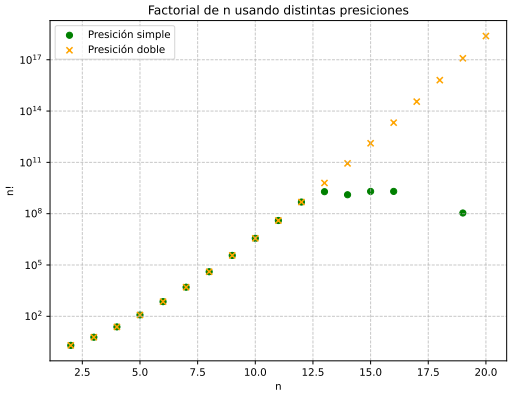


Figura 1.4

Conclusión

El análisis de la función factorial con diferentes precisiones numéricas muestra que la precisión simple (`int32`) falla rápidamente al calcular factoriales para valores grandes de n, debido a desbordamientos numéricos. En cambio, la precisión doble (`int64`) permite manejar factoriales de valores mayores sin errores hasta un límite más alto. La Figura (Figura 1.4) evidencia estas diferencias, destacando la importancia de elegir el tipo de dato adecuado según el rango esperado de n en aplicaciones que involucren cálculos factoriales.

Capítulo 2

Diferenciación numérica

Fecha de la actividad: 28 de octubre de 2024

2.1. Cálculo y comparación de derivadas numéricas y analíticas

Primero, definimos las funciones a derivar:

```
    def f(x):
        return np.sin(x)
    def g(x):
        return np.e**x
    def t(x):
        return np.log(x)
    def r(x):
        return x**2
```

En este bloque de código, cada función representa una expresión matemática: $\sin(x)$, e^x, $\ln(x)$ y x^2, respectivamente. Luego, generamos un rango de valores para x y definimos el paso h. Calculamos las derivadas numéricas (adelantada) para cada función utilizando la aproximación $(f(x+h) - f(x))/h$, y también las derivadas analíticas correspondientes:

```
    x = np.linspace(0.01, 2, 1000)
    h = 0.01

    # Derivadas numéricas
    def rDsin(x):
        return (f(x+h) - f(x)) / h
    def rDeuler(x):
        return (g(x+h) - g(x)) / h
    def rDln(x):
        return (t(x+h) - t(x)) / h
    def rDcuad(x):
        return (r(x+h) - r(x)) / h

    # Derivadas analíticas
    def d_f(x):
        return np.cos(x)
    def d_g(x):
        return np.e**x
```

```
19      def d_t(x):
20          return 1/x
21      def d_r(x):
22          return 2*x
```

Para visualizar los resultados, en las Figuras 2.1 se muestran las derivadas numéricas y analíticas para cada función. Notemos que la derivada numérica adelantada es relativamente sencilla de implementar, pero presenta menor precisión. Escoger el valor adecuado de h es crucial: si es demasiado pequeño, la derivada adelantada puede presenta problemas, afectando la estabilidad de los resultados.

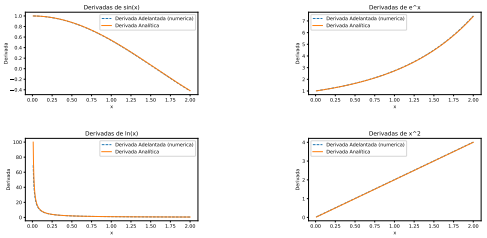


Figura 2.1: Comparacion derivadas analitica v/s numérica.

Finalmente, calculamos el error relativo entre las derivadas numéricas y analíticas de cada función mediante la fórmula:

```
1       error_rDsin = np.abs((Dsin - rDsin(x))) / np.abs(Dsin)
2       error_rDeuler = np.abs((Deuler - rDeuler(x))) / np.abs(Deuler)
3       error_rDlog = np.abs((Dln - rDln(x))) / np.abs(Dln)
4       error_rDcuad = np.abs((Dcuad - rDcuad(x))) / np.abs(Dcuad)
```

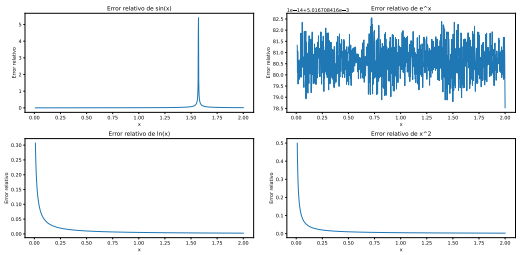


Figura 2.2: Grafico de errores.

Para funciones suaves y de comportamiento estable como e^x o x^2, la derivada adelantada proporciona una buena aproximación, aunque con menor precisión en comparación a otros métodos más complejos. Sin embargo, en funciones como $\ln(x)$ cercanas a sus límites de dominio, esta derivada suele producir errores mayores, ya que la función tiende a variar rápidamente en dichos puntos, afectando la estabilidad y exactitud de la aproximación.

En resumen, la derivada adelantada es fácil de implementar y útil para funciones suaves, pero su precisión se ve comprometida para valores extremos de h y para funciones con comportamiento abrupto en su dominio.

Conclusión

La derivada numérica adelantada es una herramienta útil y sencilla de implementar para aproximar derivadas, especialmente en funciones suaves y estables. Sin embargo, su precisión depende fuertemente de la elección adecuada del paso h, mostrando limitaciones para funciones con cambios abruptos o cercanas a los límites de su dominio.

2.2. Método de diferencias finas unilaterales de segundo orden para aproximar la derivada

Para hallar una aproximación de segundo orden para $f'(x)$ mediante diferencias finitas unilaterales, partimos de las expansiones de Taylor de $f(x+h)$ y $f(x+2h)$:

1. Expansión de $f(x+h)$:

$$f(x+h) = f(x) + f'(x)\cdot h + \frac{f''(x)}{2}\cdot h^2 + \frac{f'''(x)}{6}\cdot h^3 + \mathcal{O}(h^4).$$

2. Expansión de $f(x+2h)$:

$$f(x+2h) = f(x) + f'(x)\cdot (2h) + \frac{f''(x)}{2}\cdot (2h)^2 + \frac{f'''(x)}{6}\cdot (2h)^3 + \mathcal{O}(h^4).$$

Al simplificar, obtenemos:

$$f(x+2h) = f(x) + 2f'(x)h + 2f''(x)h^2 + \frac{4f'''(x)}{3}h^3 + \mathcal{O}(h^4).$$

A continuación, vamos a combinar linealmente los valores $f(x)$, $f(x+h)$ y $f(x+2h)$ para aproximar $f'(x)$ de manera que el error sea de orden h^2. Consideremos la combinación lineal:

$$f'(x) \approx f(x) + bf(x+h) + cf(x+2h).$$

y buscamos elegir a, b y c de modo que los términos de orden cero y segundo desaparezcan, dejando solo el de primer orden, con error de $\mathcal{O}(h^2)$.

A partir de la expansión de Taylor, obtenemos el siguiente sistema de ecuaciones:

$$\begin{cases} f(x)\text{: } a+b+c=0 \\ f'(x)\text{: } b+2c=1 \\ f''(x)\text{: } \frac{b}{2}+2c=0 \end{cases}$$

Solucionando este sistema se encuentra que: $a=-3$, $b=4$, $c=-1$. Así, la fórmula de aproximación para $f'(x)$ con segundo orden de precisión es:

$$f'(x) \approx \frac{-3f(x)+4f(x+h)-f(x+2h)}{2h}.$$

El error principal en esta fórmula es de orden $\mathcal{O}(h^2)$, lo cual confirma que esta es una diferencia finita unilateral precisa de segundo orden para la derivada de $f(x)$,

Conclusión

El método de diferencias finitas unilaterales de segundo orden permite obtener una aproximación más precisa para $f'(x)$ al incluir términos adicionales de la expansión de Taylor, reduciendo el error principal a $\mathcal{O}(h^2)$. La formula resultante es especialmente útil cuando no se dispone de valores de $f(x-h)$, como en el caso de fronteras o bordes. Esto la hace una herramienta eficaz para problemas con datos unilaterales y requisitos de mayor precisión.

2.3. Estimación de la segunda derivada utilizando diferencias centradas

Se utiliza una fórmula de diferencias finitas centradas para estimar la segunda derivada de una función $f(x)$ en un punto dado x. La aproximación de la segunda derivada $f''(x)$ con diferencias centradas de segundo orden se basa en los valores de la función en los puntos $x+h$, x, y $x-h$, donde h es un pequeño incremento. La fórmula general para esta estimación es:

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

Además debemos saber la derivada analitica, que está dada por:

$$f''(x) = \frac{d^2}{dx^2} f(x)$$

Luego, aplicando estas fórmulas, se estima la segunda derivada de la función $y = \sqrt{x}$ en los puntos $x = 0.1$, $x = 1$ y $x = \frac{\pi}{2}$. Estos puntos han sido seleccionados para evaluar cómo varía la precisión de la derivada numérica en diferentes valores de x, especialmente en valores pequeños y moderados. En python nos queda:

```
# Aproximación de la segunda derivada usando diferencias centradas
deriv2_centrada = (f(x + h) + f(x - h) - 2 * f(x)) / (h ** 2)

# Derivada analítica de la función f(x) = sqrt(x)
deriv2_analitica = -0.25 * (x ** (-3 /2))
```

Una vez calculadas las derivadas numéricas, se determina el error de esta estimación comparando con la derivada analítica de la función $y = \sqrt{x}$, cuya segunda derivada se obtiene de forma exacta. El error numérico se calcula como la diferencia entre la derivada numérica y la analítica en cada punto. Posteriormente, se grafica este error en función de x para visualizar la precisión del método en los diferentes puntos.

```
# Cálculo del error absoluto
error_absoluto = np.abs(deriv2_centrada - deriv2_analitica)

# Cálculo del error relativo
error_relativo = error_absoluto / np.abs(deriv2_analitica)

# Identificación del valor de h donde el error absoluto es mínimo
error_min = np.argmin(error_absoluto)
```

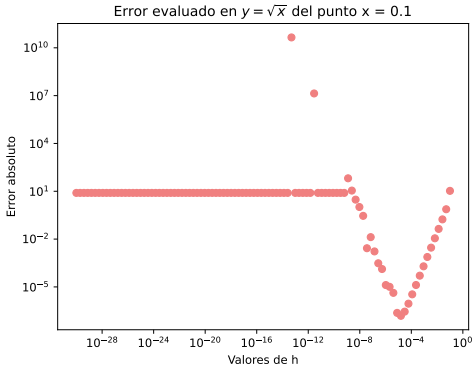
Finalmente, el error relativo de la estimación se calcula para cada punto de evaluación como la relación entre el error absoluto y el valor exacto de la segunda derivada. Este valor cuantifica la precisión del método en términos relativos y permite entender mejor la magnitud del error en relación con el valor verdadero de la derivada.

Los errores relativos en los diferentes puntos de evaluación son los siguientes

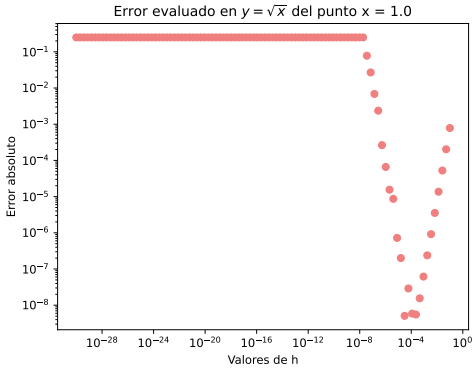
$$\text{Error relativo en } x = 1.0: \quad 3.053855508833412 \times 10^{-5}$$

$$\text{Error relativo en } x = 0.1: \quad 1.555676143930466 \times 10^{-5}$$

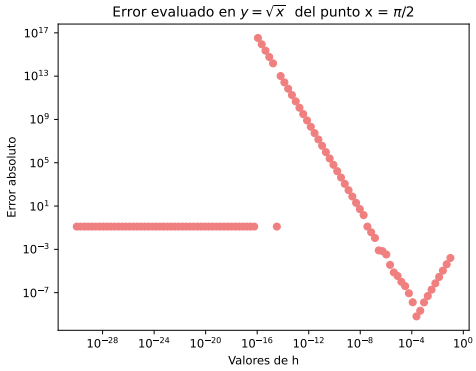
$$\text{Error relativo en } x = \frac{\pi}{2}: \quad 0.0002310129700083158$$



(a) Comparación en $x = 0.1$



(b) Comparación en $x = 1$



(c) Comparación en $x = \frac{\pi}{2}$

Figura 2.3: Comparación de derivadas analítica vs. numérica en distintos puntos de x.

Conclusión

El análisis realizado para la estimación de la segunda derivada de la función $y = \sqrt{x}$ utilizando diferencias centradas demostró que este método proporciona resultados precisos, con errores relativos pequeños en los puntos evaluados.

En particular, los errores relativos en los puntos $x = 1$ y $x = 0.1$ son notablemente bajos ($\sim 10^{-5}$), indicando una alta precisión en estas regiones. Sin embargo, en $x = \frac{\pi}{2}$, aunque el error relativo es ligeramente mayor ($\sim 10^{-4}$), sigue siendo aceptable para aplicaciones donde se permite una menor exactitud.

Los resultados muestran que el método de diferencias centradas es efectivo y confiable para la estimación de derivadas segundas, especialmente en valores de x cercanos al origen o moderados. La precisión puede depender de factores como el tamaño del incremento h y la naturaleza de la función, lo que sugiere que una elección adecuada de h es fundamental para minimizar el error.

2.4. Derivada centrada por medio del promedio de las derivadas adelanta y retrasada

Para hallar el orden de error del promedio de las derivadas, debemos expandir en serie de taylor, de modo que

$$f(x+h) \approx f(x) + f'(x)h + \frac{f''(x)h^2}{2!} + \frac{f'''(x)h^3}{3!} \dots \tag{2.1}$$

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} - \frac{f''(x)h}{2} - \frac{f'''(x)h^2}{6} \dots \tag{2.2}$$

$$f(x-h) \approx f(x) - f'(x)h + \frac{f''(x)h^2}{2!} - \frac{f'''(x)h^3}{3!} \dots \tag{2.3}$$

$$f'(x) \approx \frac{-f(x-h) + f(x)}{h} + \frac{f''(x)h}{2} - \frac{f'''(x)h^2}{6} \dots \tag{2.4}$$

Luego, promediamos (2.2) con (2.4)

$$f'(x) \approx \frac{1}{2}\left(\frac{f(x+h) - f(x)}{h} - \frac{f''(x)h}{2} - \frac{f'''(x)h^2}{6} + \frac{-f(x-h) + f(x)}{h} + \frac{f''(x)h}{2} - \frac{f'''(x)h^2}{6}\right) \tag{2.5}$$

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} - \frac{f'''(x)h^2}{6} \tag{2.6}$$

Obteniendo así la derivada centrada. Esta expresión es de orden h^2, lo que significa que el error de esta aproximación es del orden $O(h^2)$.

Al obtener un error de orden h^2, la aproximación de la derivada centrada ofrece una mayor precisión en comparación con las derivadas adelantada y retrasada, cuyo error es de orden h. Esto implica que el error decae más rápidamente a medida que se reduce el valor de h, lo que nos permite obtener estimaciones más precisas de la derivada.

Conclusión

En esta sección se dedujo la derivada centrada utilizando el promedio de las derivadas adelantada y retrasada, mostrando que esta aproximación tiene un error de orden $O(h^2)$. Esto la hace significativamente más precisa que las derivadas adelantada y retrasada, cuyos errores son de orden $O(h)$.

El resultado demuestra que la simetría en la construcción de la derivada centrada contribuye a cancelar los términos lineales del error, permitiendo que el término dominante en el error sea cuadrático. Por lo tanto, esta metodología resulta particularmente útil cuando se busca precisión en la estimación de derivadas numéricas, especialmente para valores pequeños de h.

2.5. Derivadas numéricas usando series de Taylor

Las derivadas numéricas se pueden aproximar utilizando las series de Taylor, que expanden $f(x)$ alrededor de un punto x. A continuación, se deducen las fórmulas para la derivada adelantada, retrasada y centrada, junto con el análisis del orden de error.

a) Derivada adelantada

La serie de Taylor de $f(x+h)$ alrededor de x es:

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots$$

Reorganizando para obtener la derivada:

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{f''(x)}{2}h - \frac{f'''(x)}{6}h^2 - \dots$$

Esta es la fórmula de la derivada adelantada con un error de orden $O(h)$. Es decir, el error es proporcional a h.

b) Derivada retrasada

La serie de Taylor de $f(x-h)$ alrededor de x es:

$$f(x-h) = f(x) - f'(x)h + \frac{f''(x)}{2!}h^2 - \frac{f'''(x)}{3!}h^3 + \dots$$

Reorganizando para obtener la derivada:

$$f'(x) = \frac{f(x) - f(x-h)}{h} + \frac{f''(x)}{2}h - \frac{f'''(x)}{6}h^2 + \dots$$

Esta es la fórmula de la derivada retrasada, también con un error de orden $O(h)$.

c) Derivada centrada

La serie de Taylor de $f(x+h)$ y $f(x-h)$ alrededor de x son:

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots$$

$$f(x-h) = f(x) - f'(x)h + \frac{f''(x)}{2!}h^2 - \frac{f'''(x)}{3!}h^3 + \dots$$

Sumando estas dos ecuaciones:

$$f(x+h) + f(x-h) = 2f(x) + f''(x)h^2 + \frac{f''''(x)}{12}h^4 + \dots$$

Dividiendo entre $2h$:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{f'''(x)}{6}h^2 + \dots$$

Esta es la fórmula de la derivada centrada con un error de orden $O(h^2)$, que es más precisa que las derivadas adelantada y retrasada.

Conclusión

En este análisis se presentó la deducción de las fórmulas para la derivada adelantada, retrasada y centrada mediante series de Taylor, destacando sus órdenes de error. Se concluye que, aunque los métodos adelantado y retrasado tienen un error proporcional al tamaño del paso h, el método centrado ofrece mayor precisión al reducir el error a un orden proporcional a h^2.

Esta mejora en el método centrado se debe a la simetría en la aproximación, lo que permite cancelar los términos lineales del error. Por lo tanto, el método centrado es la opción más adecuada para estimaciones numéricas de derivadas cuando se busca un equilibrio entre precisión y simplicidad en los cálculos.

2.6. Estimación del valor óptimo de h para derivadas numéricas

Al aproximar derivadas y otros operadores, inevitablemente introducimos dos tipos principales de errores: el error de truncamiento y el error computacional. El error de truncamiento ocurre debido a la aproximación de una serie infinita por una suma finita, mientras que el error computacional se introduce por la precisión limitada en los cálculos. Ambos errores afectan la precisión de los métodos numéricos, y en este trabajo se analiza cómo se combinan y cómo se puede optimizar el valor de h para minimizar ambos errores en la estimación de derivadas.

Errores en derivadas numéricas

Combinación de errores: error total

El error total en una aproximación numérica se obtiene sumando los errores de truncamiento y computacional. El error total en el método hacia adelante es la suma de estos dos errores:

$$Error = |E_{\text{trunc}}(h)| + |E_{\text{comp}}(h)|$$

Error de truncamiento

El error de truncamiento es el error que se introduce cuando una serie infinita se aproxima por un número finito de términos. En el caso de la derivada numérica, este error se introduce cuando usamos aproximaciones para la derivada, como los métodos de diferencia hacia adelante o centrada.

Por ejemplo, expandamos la derivada adelantada por series de Taylor

$$\frac{f(x_0+h)-f(x_0)}{h}$$

Usando la expansión en serie de Taylor para $f(x_0+h)$ alrededor de x_0:

$$f(x_0+h) = f(x_0) + hf'(x_0) + \frac{h^2}{2}f''(x_0)$$

Despejando $f'(x_0)$:

$$f'(x_0) = \frac{f(x_0+h)-f(x_0)}{h} - \frac{h}{2}f''(\xi) \tag{2.7}$$

El término $\frac{h}{2}f''(\xi)$ es el error de truncamiento que aparece debido a la aproximación de la expansión de Taylor truncada. Este error es de orden $O(h)$, lo que significa que disminuye linealmente con h.

Por lo tanto, el error de truncamiento en el método hacia adelante es proporcional a h, y disminuye a medida que reducimos h.

Error computacional

El error computacional es el error introducido debido a la precisión limitada de los cálculos en la computadora. Este error se modela como un error relativo que afecta a los valores de la función evaluada en cada punto. Supongamos que la función $f(x)$ se calcula con un error relativo ϵ, de modo que la función computada $\bar{f}(x)$ es:

$$\bar{f}(x) = f(x)(1+\epsilon)$$

Donde ϵ es pequeño y depende de la precisión de la máquina, por lo que generalmente $|\epsilon| \ll 1$.

Cuando utilizamos esta versión perturada de la función $\bar{f}(x)$ para aproximar la derivada, introducimos errores adicionales. Consideremos el método hacia adelante para la derivada, pero ahora utilizando $\bar{f}(x)$, Usando la expansión en serie de Taylor de $f(x_0 + h)$:

Sustituyendo la expresión $\bar{f}(x) = f(x)(1 + \epsilon)$ en (2.7), obtenemos:

$$\frac{[f(x_0 + h)(1 + \epsilon_1) - f(x_0)(1 + \epsilon_2)]}{h} - \frac{h}{2}f''(\xi)$$

Sustituyendo esto en la expresión anterior:

$$f'(x) - f(x_0 + h) - f(x_0) = \frac{[f(x_0 + h)\epsilon_1 - f(x_0)\epsilon_2]}{h} - \frac{h}{2}f''(\xi)$$

Usando desigualdad triangular tenemos:

$$|Error| \leq \left|\frac{f(x_0 + h)\epsilon_1}{h}\right| + \left|\frac{f(x_0)\epsilon_2}{h}\right| + \left|\frac{h}{2}f''(\xi)\right|$$

Ahora bien, podemos definir una cota superior que llamaremos τ, donde $\epsilon_1 \leq \tau$ y $\epsilon_2 \leq \tau$, además sabemos que $O(h) = \frac{f(\xi)h}{2}$ como ξ es un valor muy pequeño podemos hacer la siguiente aproximacion $f(\xi) \approx f(x_0)$, ademas el paso $h < 1$ por lo que $f(x_0 + h) \approx f(x_0)$

$$|Error| \leq \left|\frac{2f(x_0)\tau}{h}\right| + \left|\frac{h}{2}f''(x_0)\right|$$

Ahora bien, para buscar el h optimo debemos derivar h e igualar a 0:

$$\frac{\partial E}{h} = -4\left|\frac{f(x_0)\tau}{h^2}\right| + \left|\frac{f''(x_0)}{2}\right|$$

$$-2\left|\frac{f(x_0)\tau}{h^2}\right| + \left|\frac{f''(x_0)}{2}\right| = 0$$

$$\left|\frac{f''(x_0)}{2}\right| = 2\left|\frac{f(x_0)\tau}{h^2}\right|$$

$$h = \sqrt{\left|4\frac{f(x_0)\tau}{f''(x_0)}\right|}.$$

Ahora bien, Como la derivada adelanta y retrasada tienen el mismo orden de h, el valor optimo de este será el mismo.

Por ultimo, se puede obtener el valor optimo de la derivada centrada de forma anloga:

$$h = \left(\frac{6\,|f(x_0)\tau|}{|f^3(x_0)|}\right)^{3/2}$$

Conclusión

En este trabajo se analizó la interacción entre el error de truncamiento y el error computacional en la estimación numérica de derivadas utilizando métodos de diferencias finitas. Se determinó que el valor óptimo de h puede calcularse para minimizar el error total, destacando que las derivadas adelantada, retrasada y centrada tienen expresiones específicas para este valor.

El análisis resalta la importancia de equilibrar ambos errores, ya que valores muy pequeños de h amplifican el error computacional, mientras que valores grandes incrementan el error de truncamiento. Por tanto, la correcta selección de h es clave para obtener estimaciones más precisas.

Además, se confirmó que los métodos centrados presentan una ventaja significativa al reducir el error de truncamiento, lo que los hace preferibles en diversas aplicaciones numéricas, siempre considerando las características de la función involucrada.

Capítulo 3

Integrales

Fecha de la actividad: 30 de octubre de 2024

3.1. Obtención de pesos óptimos para la aproximación de Integrales con exactitud en polinomios de grado 2

Veamos como podemos obtener los pesos optimos para lograr una buena aproximación, se plantea la siguiente integral:

$$\int_{-1}^{1} f(x)\sin(\pi x/2) = \omega_{-1}f(-1) + \omega_0 f(0) + \omega_1 f(1).$$

Luego, asumamos que la integral es exacta hasta polinomios de grado 2, de modo que:

$$f(x) = 1 \tag{3.1}$$

$$f(x) = x \tag{3.2}$$

$$f(x) = x^2 \tag{3.3}$$

Luego integramos y evaluamos respectivamente (3.1) (3.2) (3.3) de modo que:

$$\int_{-1}^{1} \sin(\frac{\pi x}{2})dx = 0 = \omega_{-1} + \omega_0 + \omega_1$$

$$\int_{-1}^{1} x\sin(\frac{\pi x}{2})dx = \frac{8}{\pi^2} = -\omega_{-1} + \omega_1$$

$$\int_{-1}^{1} x^2\sin(\frac{\pi x}{2})dx = 0 = \omega_{-1} + \omega_1$$

Con los resultados obtenidos podemos plantear un sistema de ecuaciones soluble ya que tenemos tres incognitas y tres ecuaciones

$$\begin{cases} \omega_{-1} + \omega_0 + \omega_1 = 0 \\ -\omega_{-1} + \omega_1 = \frac{8}{\pi^2} \\ \omega_{-1} + \omega_1 = 0 \end{cases}$$

Resolviendo este sistema se llega a los siguientes resultados:

$$\omega_{-1} = -\frac{4}{\pi^2}, \quad \omega_0 = 0, \quad \omega_1 = \frac{4}{\pi^2}.$$

Por lo tanto, la mejor aproximación se obtiene mediante los pesos obtenidos.

3.2. Determinación de pesos óptimos para la integración de polinomios de grado 3 con términos derivativos

Encontremos los valores de los pesos, asumiendo que esta regla es exacta para polinomios de hasta grado 3, asi tiene que:

$$f(x) = 1 \tag{3.4}$$
$$f(x) = x \tag{3.5}$$
$$f(x) = x^2 \tag{3.6}$$
$$f(x) = x^3 \tag{3.7}$$

Debemos tener cuidado con (3.4), (3.5), (3.6) y (3.7), ya que ω_2 y ω_3 acompañan a f derivada y deben evaluarse correctamente en cada caso. Integramos y evaluamos, así se obtiene:

$$\int_0^1 dx = 1 = \omega_0 + \omega_1$$
$$\int_0^1 xdx = \frac{1}{2} = \omega_1 + \omega_2 + \omega_3$$
$$\int_0^1 x^2 dx = \frac{1}{3} = \omega_1 + 2\omega_3$$
$$\int_0^1 x^3 dx = \frac{1}{4} = \omega_1 + 3\omega_3$$

Luego, planteamos un sistema de ecuaciones para obtener los valores de los pesos, de modo que:

$$\begin{cases} \omega_0 + \omega_1 = 1 \\ \omega_1 + \omega_2 + \omega_3 = \frac{1}{2} \\ \omega_1 + 2\omega_3 = \frac{1}{3} \\ \omega_1 + 3\omega_3 = \frac{1}{4} \end{cases}$$

Finalmente, resolviendo el sistema obtenemos los siguientes valores:

$$\omega_0 = \frac{1}{2}$$
$$\omega_1 = \frac{1}{2}$$
$$\omega_2 = \frac{1}{12}$$
$$\omega_3 = \frac{-1}{12}$$

Error del metodo

Para calcular el error se debe tener en consideración esta ecuacion:

$$Error = I_{exacta} - S \tag{3.8}$$

Hay que tener en consideracion que se debe expandir en series de Taylor tanto la integral como la sumatoria Primero expandamos la integral en torno a $x = 0$

$$f(x) = f(0) + f'(0)x + \frac{f''(0)x^2}{2} + \frac{f^3(0)x^3}{6} + \frac{f^4(\xi)h^4}{24} \tag{3.9}$$

Ahora integramos

$$I_{exacta} = \int_0^1 f(x)dx = f(0) + \frac{f'(0)}{2} + \frac{f''(0)}{6} + \frac{f^3(0)}{24} + \frac{f^4(\xi)h^5}{120} \tag{3.10}$$

Ahora, la sumatoria esta dada por:

$$S = \sum_{i=0}^{n} w_i f(x_i) = 1/2[f(0) - f(1)] + 1/12[f'(0) - f'(1)] \tag{3.11}$$

Debemos dejar todo en temrinos de f(0) por conveniencia, por lo que usamos series de taylor evaluamos los puntos en torno a x=0 y asi se obtiene lo siguiente:

$$S = \sum_{i=0}^{n} w_i f(x_i) = f(0) + \frac{f'(0)}{2} + \frac{f''(0)}{6} + \frac{f^3(0)}{24} + \frac{f^4(\xi)h^5}{144} \tag{3.12}$$

Ahora reemplazamos (3.10) y (3.12) en (3.8) se tiene:

$$Error = \frac{f^4(\xi)h^5}{720} \tag{3.13}$$

Conclusión

En este trabajo, hemos determinado los pesos óptimos para la integración de polinomios de hasta grado 3 utilizando una regla de cuadratura. A partir de las condiciones de exactitud de la regla para los polinomios $f(x) = 1$, $f(x) = x$, $f(x) = x^2$, y $f(x) = x^3$, se planteó un sistema de ecuaciones para obtener los valores de los pesos ω_0, ω_1, ω_2, y ω_3.

Las ecuaciones resultantes fueron las siguientes:

- $\omega_0 + \omega_1 = 1$
- $\omega_1 + \omega_2 + \omega_3 = \frac{1}{2}$
- $\omega_1 + 2\omega_3 = \frac{1}{3}$
- $\omega_1 + 3\omega_3 = \frac{1}{4}$

Tras resolver este sistema, se obtuvieron los siguientes valores para los pesos:

$$\begin{aligned} \omega_0 &= \tfrac{1}{2} \\ \omega_1 &= \tfrac{1}{2} \\ \omega_2 &= \tfrac{1}{12} \\ \omega_3 &= \tfrac{-1}{12} \end{aligned}$$

Estos valores corresponden a los pesos óptimos que permiten integrar de manera exacta polinomios hasta el grado 3, considerando las condiciones derivadas de la exactitud de la regla de cuadratura. Los resultados obtenidos indican que la regla de cuadratura propuesta es efectiva para este tipo de polinomios y que la elección adecuada de los pesos permite garantizar la exactitud de la integración.

3.3. Error del metodo trapezoidal

Antes de introducirnos a problemas donde se use este metodo, debemos saber su error para ver que tan preciso es. A continuación se presenta la aproximación:

$$I_{aprox} = \frac{(b-a)([f(a)+f(b)])}{2} \tag{3.14}$$

sabemos por la ecuacion (3.8) que se debe expandir ambos terminos en series de taylor, por conveniencia se expandira en torno a x=a hasta la segunda derivada y se evaluará en f(b) para así tener solo un termino en comun

$$f(b) = f(a) + f'(a)(b-a) + \frac{f''(\xi)(b-a)^2}{2} \tag{3.15}$$

Luego reemplazamos (3.15) en (3.16) se tiene

$$I_{aprox} = \frac{(b-a)}{2}\left[f(a) + f(a) + f'(a)(b-a) + \frac{f''(\xi)(b-a)^2}{2}\right] \tag{3.16}$$

por otro lado la integral exacta esta dada por:

$$I_{exacta} \int_a^b f(x)dx \tag{3.17}$$

expandimos en series de taylor en torno x=a hasta el segundo termino e integramos

$$I_{exacta} = f(a)(b-a) + \frac{f'(a)(b-a)^2}{2} + \frac{f''(\xi)(b-a)^3}{6} \tag{3.18}$$

Sumando la integral exacta con la inexacta se obtiene el error del metodo

$$Error = \frac{-f''(\xi)(b-a)^3}{12} = \frac{-f''(\xi)h^3}{12} \tag{3.19}$$

3.4. Principio del argumento de Cauchy

El principio del argumento de Cauchy indica que si $f(z)$ es una función meromorfa sin polos en una región encerrada por un contorno C, entonces

$$N = \frac{1}{2\pi i}\oint_C \frac{f'(z)}{f(z)}\,dz,$$

representa el número N de ceros de f, i.e., donde $f(z) = 0$, que están encerrados por el contorno C.

Para calcular $f(z) = z^4 - \frac{1}{2}$ dentro del disco de radio unitario $|z| < 1$. Se mostrará los mas imprtante del codigo y su respectivo grafico.

```
    # Parametrizamos el contorno C (el círculo unitario)
    theta = np.linspace(0, 2 * np.pi, 1000)
    z_contour = np.exp(1j * theta)  # Círculo unitario |z| = 1

    # Evaluamos la integral numéricamente usando el método de trapezoides
    integrand = df(z_contour) / f(z_contour)
    integral_value = np.trapz(integrand, z_contour)
```

Notemos que el numero de ceros se saca usando el principo de argumento de Cauchy de modo que

```
    N_zeros = integral_value / (2j * np.pi)
```

El cero obtenido para esta funcion es ≈ 3.99

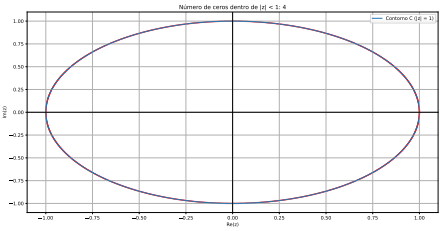


Figura 3.1: Número de ceros dentro de $|z| < 1$

Luego, Para la función $f(z) = z^4 - 5z + 1$ dentro de la región anular $1 < |z| < 2$ Tenemos:

```
    # Parametrizamos los contornos del anillo
    theta = np.linspace(0, 2 * np.pi, 1000)
    contour_inner = np.exp(1j * theta)  # Contorno interior |z| = 1
    contour_outer = 2 * np.exp(1j * theta)  # Contorno exterior |z| = 2

    # Evaluamos la integral numéricamente para ambos contornos usando el método de
    ↪  trapezoides
    integrand_inner = df_b(contour_inner) / f_b(contour_inner)
    integrand_outer = df_b(contour_outer) / f_b(contour_outer)

    # Calculamos el número de ceros dentro de cada contorno
    N_inner = integral_inner / (2j * np.pi)
    N_outer = integral_outer / (2j * np.pi)

    # El número de ceros en la región anular es la diferencia
    N_zeros_annulus = N_outer - N_inner
```

Notemos que

```
    # El número de ceros en la región anular es la diferencia
    N_zeros_annulus = N_outer - N_inner
```

Por lo que el cero es $\approx 2.99 - 1.76i$

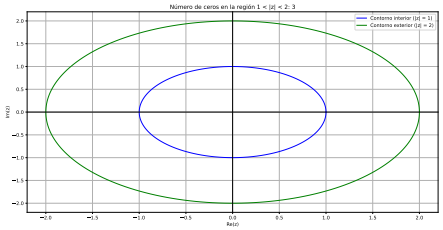


Figura 3.2: numero de ceros en la región $1 < |z| < 2$

Finalmente, para la función $f(z) = \sin z + \frac{1}{(z+i)}$ en la región cuadrada $-1 < \mathrm{Im}(z) < 1$ y $\pi(100 + \frac{1}{2}) < \mathrm{Re}(z) < \pi(200 + \frac{1}{2})$ se tiene:

```
    # Parámetros para el contorno cuadrado
    n_points = 1000
    x_left = np.pi * (100 + 0.5)
    x_right = np.pi * (200 + 0.5)
    y_bottom = -1
    y_top = 1

    # Definir los contornos C1, C2, C3 y C4
    x = np.linspace(x_left, x_right, n_points)
    y = np.linspace(y_bottom, y_top, n_points)
    # Sumar las integrales de los contornos
    total_integral = integral_C1 + integral_C2 + integral_C3 + integral_C4

    # Calcular el número de ceros considerando el polo en z = -i
    N_poles = 1  # Polo simple en z = -i
    N_zeros = total_integral / (2 * np.pi * 1j) + N_poles
```

Para calcular el número de ceros se define

```
    # Calcular el número de ceros considerando el polo en z = -i
    N_poles = 1  # Polo simple en z = -i
    N_zeros = total_integral / (2 * np.pi * 1j) + N_poles
```

El cero es $\approx 1 + 0.0003i$

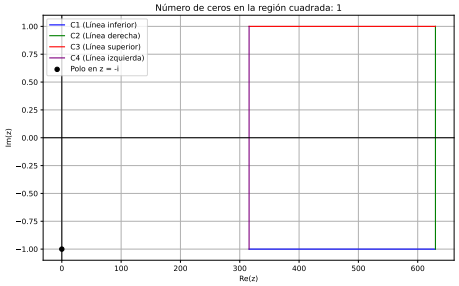


Figura 3.3: Número de ceros en la región cuadrada

Conclusión

En este trabajo hemos aplicado el principio del argumento de Cauchy para determinar el número de ceros de varias funciones en diferentes regiones, utilizando diferentes contornos de integración.

Primero, para la función $f(z) = z^4 - \frac{1}{2}$ dentro del disco unitario $|z| < 1$, hemos parametrizado el contorno C como el círculo unitario y hemos evaluado la integral numéricamente. Usando el principio del argumento, calculamos el número de ceros de la función encerrados por el contorno, lo cual se muestra en la Figura 3.1.

Luego, para la función $f(z) = z^4 - 5z + 1$ en la región anular $1 < |z| < 2$, realizamos un análisis similar, parametrizando dos contornos: uno para el círculo de radio 1 y otro para el círculo de radio 2. La diferencia de los resultados obtenidos de las integrales en ambos contornos nos permitió calcular el número de ceros en la región anular, como se ilustra en la Figura 3.2.

Finalmente, para la función $f(z) = \sin z + \frac{1}{(z+i)}$ en la región cuadrada definida por $-1 < \mathrm{Im}(z) < 1$ y $\pi(100 + \frac{1}{2}) < \mathrm{Re}(z) < \pi(200 + \frac{1}{2})$, evaluamos la integral a lo largo de los contornos cuadráticos y

tomamos en cuenta el polo en $z = -i$. El número de ceros de la función en la región cuadrada fue calculado y presentado en la Figura 3.3.

En conclusión, el principio del argumento de Cauchy nos ha permitido obtener el número de ceros de las funciones en las respectivas regiones mediante la evaluación numérica de las integrales de los cocientes $\frac{f'(z)}{f(z)}$ a lo largo de los contornos adecuados. Los resultados obtenidos para cada caso han sido consistentes con las expectativas teóricas y han proporcionado una demostración eficaz de este poderoso principio en el análisis complejo.

Agradecimientos

Agradezco al ayudante Juan Orteaga por ayudarme a realizar el ejercicio, fue muy considerado.

3.5. Aproximación de la integral de funciones tabuladas usando la regla trapezoidal

En este problema, se busca aproximar la integral de una función $f(x)$ utilizando la regla trapezoidal. Dado un conjunto de datos tabulados de la función, cuyo valor solo se conoce en puntos discretos $x_1, x_2, \ldots, x_n$, la regla trapezoidal permite estimar el área bajo la curva mediante la suma de áreas de trapecios formados entre puntos consecutivos. En este caso, se utiliza la función $f(x) = x^2$ evaluada en puntos específicos, y se aplicará la técnica de integración p ara aproximar su integral en un intervalo determinado.

Para este ejemplo usaremos los siguientes datos:

x	$y = f(x)$
0	0
1	1
2	4
3	9
4	16

Cuadro 3.1: Puntos tabulados para $f(x) = x^2$.

A continuación se menciona la parte mas relevante del codigo:

```
    #Calcula la integral de datos tabulados usando la regla trapezoidal.
    def regla_trapezoidal(x, y):
        if len(x) != len(y):
            raise ValueError("Los vectores x e y deben tener la misma longitud.")
        if not np.all(np.diff(x) > 0):
            raise ValueError("Los valores de x deben estar ordenados de manera
            ↪  estrictamente creciente.")


    # Implementación de la regla trapezoidal
    h = np.diff(x)
```

```
11      integral = np.sum(h * (y[:-1] + y[1:]) / 2)
12      return integral
```

Finalmente el valor aproximado de la integral es 22

Conclusión

En este ejercicio, hemos utilizado la regla trapezoidal para aproximar la integral de la función $f(x) = x^2$ a partir de un conjunto de datos tabulados. La técnica consiste en estimar el área bajo la curva mediante la suma de las áreas de trapecios formados entre puntos consecutivos de los datos. Los puntos tabulados para $f(x) = x^2$ fueron presentados en la Tabla 3.1, y la implementación de la regla trapezoidal se detalló en el código.

Al aplicar la regla trapezoidal a los datos tabulados, obtuvimos un valor aproximado para la integral, que resultó ser 22. Este valor refleja la suma de los trapecios que aproximan el área bajo la curva de la función en el intervalo considerado. La técnica de la regla trapezoidal proporciona una forma eficiente y sencilla de aproximar integrales cuando los valores de la función solo están disponibles en puntos discretos.

El resultado obtenido es una buena aproximación a la integral exacta de la función, demostrando la efectividad de la regla trapezoidal en la integración numérica de funciones tabuladas.

Capítulo 4

Ecuaciones Diferenciales Ordinarias

Fecha de la actividad: 30 de noviembre de 2024

4.1. Solución Numérica y Comparación de la Función de Bessel de Primer Tipo en el Intervalo $0 < x \leq 20$

La ecuación de Bessel de orden 0 en su forma original es una ecuación diferencial de segundo orden:

$$J_0''(x) + \frac{1}{x}J_0'(x) + J_0(x) = 0$$

Para convertirla en un sistema de ecuaciones de primer orden, definimos el vector de estado $r(x)$ como:

$$r(x) = \begin{bmatrix} J_0(x) \\ J_0'(x) \end{bmatrix}$$

Esto nos lleva a las siguientes ecuaciones de primer orden:

$$\frac{d}{dx}J_0(x) = J_0'(x)$$

$$\frac{d}{dx}J_0'(x) = -J_0(x) - \frac{1}{x}J_0'(x)$$

El sistema de ecuaciones en álgebra lineal es:

$$\frac{d}{dx}\begin{bmatrix} J_0(x) \\ J_0'(x) \end{bmatrix} = \begin{bmatrix} J_0'(x) \\ -J_0(x) - \frac{1}{x}J_0'(x) \end{bmatrix}$$

A continuacion se presenta el codigo en python implementando la solucion por medio de Runge-kutta4

```python
# Sistema de ecuaciones de Bessel de orden 0
def f(r, x):
    J0, J0_dx = r # ([J0(x), J0'(x)])
    dJ0_2dx = -J0 - (1/x) * J0_dx  # Ecuación diferencial de Bessel
```

```
    return np.array([J0_dx, dJ0_2dx])

# Condiciones iniciales
r0 = [1.0, 0.0]  #J0(0) = 1, J0'(0) = 0

# Generar los valores de x
x_valores = np.arange(1e-5, 20, 0.05)  #va de "0" hasta 20 con paso de 0.05

# Resolver la ecuación diferencial usando Runge-Kutta de 4to orden
solucion = RungeKutta4(f, r0, x_valores) #RungeKutta4(f,r,x) o RungeKutta4(f,r,t)

# Obtener J0(x) de la solución
J0_numerico = solucion[:, 0] #(si fuera [:,1] me entregaría las derivadas de la solucion)
```

Luego, integramos con el metodo de trapecios, de modo que

```
# Definir la función de integrando
def integrando(x, phi):
    return np.cos(x * np.sin(phi))

# Generar los valores de x y phi
x_valores = np.arange(1e-5, 20, 0.05)  # Valores de x
phi_valores = np.linspace(0, np.pi, 1000)  # Intervalo para phi de [0, pi]

# Evaluar la función integrando para cada combinación de x y phi
resultados = []

# Evaluar la función integrando para cada valor de x y phi
for x in x_valores:
    integrales_x = []  # Lista para almacenar la integral de cada x con respecto a phi
    for phi in phi_valores:
        integrales_x.append(integrando(x, phi))
    resultados.append(integrales_x)


# Calcular la integral usando la regla trapezoidal para cada valor de x
J0_integral = np.trapz(resultados, phi_valores, axis=1)  # Integrar sobre phi para cada x
```

De esta forma, calculamos la integral y obtenemos la solución en el intervalo $0 < x \leq 20$.

Análisis del error relativo entre la solución numérica e integral

El error relativo entre la solución numérica y la integral se calculó de modo:

$$\text{Error relativo} = \frac{|J_0(x)_{\text{numerico}} - J_0(x)_{\text{exacto}}|}{|J_0(x)_{\text{numerico}}|}.$$

El siguiente código calcula y grafica el error relativo:

```
# Error relativo
error_relativo = np.abs((J0_numerico - J0_integral) / J0_integral)
```

Gráficos

En esta sección se presentan los gráficos que muestran los resultados obtenidos de la solución de la ecuación de Bessel de primer tipo. Los gráficos incluyen tanto la solución numérica como la integral y el error.

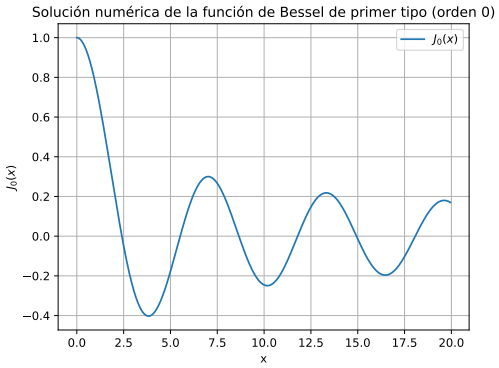


Figura 4.1: Solución numérica de la ecuación de Bessel de primer tipo utilizando el método de Runge-Kutta de cuarto orden.

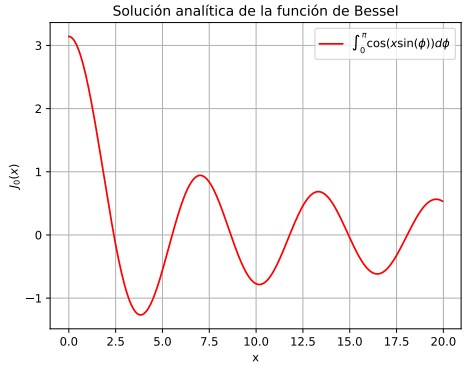


Figura 4.2: Solución integral de la ecuación de Bessel de primer tipo obtenida utilizando la regla trapezoidal.

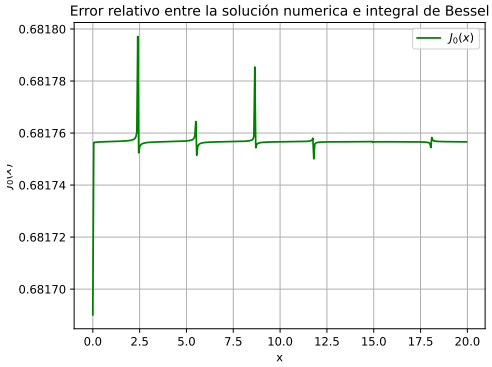


Figura 4.3: Error relativo entre la solución numérica e integral de la ecuación de Bessel de primer tipo.

Conclusión

En este trabajo se resolvió la ecuación de Bessel de primer tipo utilizando dos enfoques numéricos. Primero, se empleó el método de Runge-Kutta de cuarto orden para obtener una solución aproximada en el intervalo $0 < x \leq 20$. Luego, se calculó la solución integral mediante la regla trapezoidal y se compararon ambos resultados.

Los gráficos presentados muestran la concordancia entre ambos métodos, aunque se observó un pequeño error relativo, el cual se analizó en detalle. El método de Runge-Kutta es eficiente para obtener soluciones numéricas precisas, mientras que la integral proporciona una aproximación útil desde una perspectiva analítica.

La comparación entre ambas soluciones y el análisis del error relativo confirma la validez de ambos enfoques para resolver la ecuación de Bessel, demostrando que ambos métodos pueden ser utilizados con éxito según las necesidades del problema.

4.2. Demostración del error local del método de Runge-Kutta de cuarto orden

El objetivo de este problema es demostrar el orden del error local del método de Runge-Kutta de cuarto orden, el cual está dado por $O(\Delta t^5)$. El método es definido por la siguiente fórmula recursiva:

$$K_1 = \Delta t f(\mathbf{x}_n), \tag{4.1}$$

$$K_2 = \Delta t f\left(\mathbf{x}_n + \frac{1}{2}K_1\right), \tag{4.2}$$

$$K_3 = \Delta t f\left(\mathbf{x}_n + \frac{1}{2}K_2\right), \tag{4.3}$$

$$K_4 = \Delta t f\left(\mathbf{x}_n + K_3\right), \tag{4.4}$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4). \tag{4.5}$$

Donde $f(\mathbf{x})$ corresponde al lado derecho de la ecuación diferencial $\mathbf{x}'(t) = f(\mathbf{x})$.

Expansión de la solución exacta en serie de Taylor

Para demostrar el orden del error local del método de Runge-Kutta de cuarto orden, expandimos la solución exacta $\mathbf{x}(t_{n+1})$ en una serie de Taylor alrededor de t_n:

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \Delta t\mathbf{x}'(t_n) + \frac{\Delta t^2}{2}\mathbf{x}''(t_n) + \frac{\Delta t^3}{6}\mathbf{x}'''(t_n) + \frac{\Delta t^4}{24}\mathbf{x}^{(4)}(t_n) + O(\Delta t^5). \tag{4.6}$$

Donde las derivadas son las siguientes:

$$\mathbf{x}'(t_n) = f(t_n, \mathbf{x}_n),$$

$$\mathbf{x}''(t_n) = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial \mathbf{x}} f,$$

$$\mathbf{x}'''(t_n) = \frac{\partial^2 f}{\partial t^2} + 2\frac{\partial^2 f}{\partial t \partial \mathbf{x}} f + \frac{\partial^2 f}{\partial \mathbf{x}^2} f^2 + \frac{\partial f}{\partial \mathbf{x}} f',$$

$$\mathbf{x}^{(4)}(t_n) = \frac{\partial^3 f}{\partial t^3} + 3\frac{\partial^3 f}{\partial t^2 \partial \mathbf{x}} f + 3\frac{\partial^3 f}{\partial t \partial \mathbf{x}^2} f^2 + \frac{\partial^3 f}{\partial \mathbf{x}^3} f^3 + 3\frac{\partial^2 f}{\partial t \partial \mathbf{x}} f' + 3\frac{\partial^2 f}{\partial \mathbf{x}^2} f f' + \frac{\partial f}{\partial \mathbf{x}} f''.$$

Cálculo y expansión de K_i en serie de Taylor alrededor de t_n

El método de Runge-Kutta de cuarto orden se puede expresar como:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4),$$

donde:

$$K_1 = \Delta t f(t_n, \mathbf{x}_n),$$

$$K_2 = \Delta t f\left(t_n + \frac{1}{2}\Delta t, \mathbf{x}_n + \frac{1}{2}K_1\right),$$

$$K_3 = \Delta t f\left(t_n + \frac{1}{2}\Delta t, \mathbf{x}_n + \frac{1}{2}K_2\right),$$

$$K_4 = \Delta t f\left(t_n + \Delta t, \mathbf{x}_n + K_3\right).$$

Empezamos con K_1, que depende directamente de f:

$$K_1 = \Delta t f(t_n, \mathbf{x}_n).$$

Para K_2, expandimos f en serie de Taylor alrededor de t_n:

$$K_2 = \Delta t f\left(t_n + \frac{1}{2}\Delta t, \mathbf{x}_n + \frac{1}{2}K_1\right) = \Delta t\left[f(t_n, \mathbf{x}_n) + \frac{\Delta t}{2}\frac{\partial f}{\partial t} + \frac{\Delta t}{2}\frac{\partial f}{\partial \mathbf{x}} f + O(\Delta t^2)\right].$$

Sustituyendo $K_1 = \Delta t f(t_n, \mathbf{x}_n)$:

$$K_2 = \Delta t f(t_n, \mathbf{x}_n) + \frac{\Delta t^2}{2}\frac{\partial f}{\partial t} + \frac{\Delta t^2}{2}\frac{\partial f}{\partial \mathbf{x}} f + O(\Delta t^2).$$

De manera similar, para K_3, expandimos f en serie de Taylor alrededor de t_n:

$$K_3 = \Delta t f\left(t_n + \frac{1}{2}\Delta t, \mathbf{x}_n + \frac{1}{2}K_2\right),$$

lo que da lugar a:

$$K_3 = \Delta t f(t_n, \mathbf{x}_n) + \Delta t\frac{\Delta t}{2}\frac{\partial f}{\partial t} + \Delta t\frac{\Delta t}{2}\frac{\partial f}{\partial \mathbf{x}} f + O(\Delta t^3).$$

Finalmente, para K_4, expandimos f alrededor de t_n:

$$K_4 = \Delta t f(t_n + \Delta t, \mathbf{x}_n + K_3),$$

lo que da:

$$K_4 = \Delta t f(t_n, \mathbf{x}_n) + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial t} + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial \mathbf{x}} f + O(\Delta t^3).$$

Cálculo de $\mathbf{x}_{n+1}$

Sustituyendo los valores de K_1, K_2, K_3 y K_4 en la fórmula de Runge-Kutta de cuarto orden, obtenemos:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4).$$

Expresando cada término:

$$\frac{1}{6} K_1 = \frac{1}{6} \Delta t f(t_n, \mathbf{x}_n),$$

$$\frac{2}{6} K_2 = \frac{1}{3} \Delta t f(t_n, \mathbf{x}_n) + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial t} + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial \mathbf{x}} f,$$

$$\frac{2}{6} K_3 = \frac{1}{3} \Delta t f(t_n, \mathbf{x}_n) + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial t} + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial \mathbf{x}} f + O(\Delta t^3),$$

$$\frac{1}{6} K_4 = \frac{1}{6} \Delta t f(t_n, \mathbf{x}_n) + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial t} + \Delta t \frac{\Delta t}{2} \frac{\partial f}{\partial \mathbf{x}} f + O(\Delta t^3).$$

Sumando estos términos y comparando con la expansión de Taylor de la solución exacta, se obtiene que el error local es de orden $O(\Delta t^5)$.

Conclusión

El error local del método de Runge-Kutta de cuarto orden es de orden $O(\Delta t^5)$, lo que significa que el error en un solo paso de integración es pequeño y depende de la quinta potencia del paso Δt, lo que garantiza una alta precisión del método.

4.3. Solución Analítica y Aproximación Numérica mediante el Método de Euler

Se presenta el análisis de un problema con valor inicial dado por la ecuación diferencial

$$y' = -3y \sin(t), \quad y(0) = \frac{1}{2}, \tag{4.7}$$

para el cual se determina su solución analítica mediante el método de **separación de variables** y se calcula una aproximación numérica utilizando el **método de Euler**.

Se realizará una comparación de ambas soluciones, evaluando la precisión para distintos valores de N, donde se toman particiones del intervalo $0 \leq t \leq 4\pi$ con $N = 2^8, 2^9, 2^{10}, 2^{11}$ subintervalos. Finalmente, se discutirán las ventajas y limitaciones del método numérico aplicado a este tipo de problemas.

Solución Analítica

Considerando la ecuación diferencial (4.7), donde $y' = \frac{dy}{dt}$, y multiplicando ambos miembros por $\frac{dt}{y}$, se obtiene una forma separable:

$$\frac{dy}{y} = -3\sin(t)\,dt.$$

Integrando ambos lados de la ecuación, se llega a:

$$\ln|y| = 3\cos(t) + C, \tag{4.8}$$

donde C es una constante de integración. Al despejar y mediante la función exponencial sobre la ecuación (4.8), se obtiene la expresión:

$$y(t) = e^{3\cos(t)}e^{C}. \tag{4.9}$$

Aplicando la condición inicial $y(0) = \frac{1}{2}$, se obtiene que

$$\frac{1}{2} = e^{3}e^{C}$$

Aplicando el logaritmo natural sobre ambos lados de la ecuación, se obtiene:

$$C = \ln\left(\frac{1}{2}\right) - 3. \tag{4.10}$$

Sustituyendo (4.10) en (4.9), y simplificando, se obtiene la solución analítica:

$$y(t) = \frac{1}{2e^{3}}e^{3\cos(t)}.$$

Solución Numérica: Método de Euler

Se resuelve numéricamente la ecuación diferencial (4.7) implementando el código en `Python` y utilizando el módulo `EcuacionesDiferencialesOrdinarias`. La aproximación para $y(t)$ se obtiene mediante el método de Euler con la fórmula:

$$y_{n+1} = y_n + \Delta t f(t_n, y_n),$$

donde

- $f(t, y) = -3y\sin(t)$,
- $t_n = n\Delta t$,
- $\Delta t = \frac{t_{\text{máx}} - t_{\text{mín}}}{N}$.

El problema se resuelve para $N = 2^8, 2^9, 2^{10}, 2^{11}$ y se compara con la solución analítica.

```
    def f(y, t):
        return -3 * y * np.sin(t)
    y0 = 1 / 2
    t_min, t_max = 0, 4 * np.pi
    N_values = [2**8, 2**9, 2**10, 2**11]
    for N in N_values:
        t = np.linspace(t_min, t_max, N + 1)
        y = Euler(f, r0=y0, t=t)
```

Resultados y Comparación

En la Figura 4.4 se muestran las soluciones obtenidas para cada valor de N, junto con la solución analítica. Se observa que, a medida que N aumenta, las soluciones numéricas se acercan más a la solución analítica, lo que evidencia la dependencia del método de Euler con el tamaño del paso Δt.

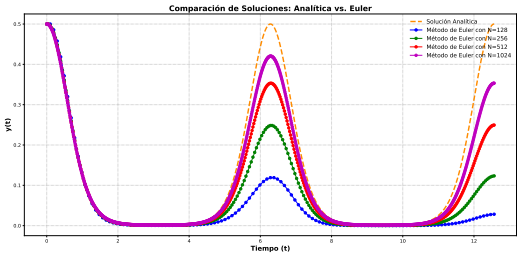


Figura 4.4: Comparación entre la solución analítica y las soluciones numéricas para diferentes valores de N.

Conclusion

En este capítulo se resolvió una ecuación diferencial ordinaria utilizando tanto un enfoque analítico como numérico. Se observó que, aunque el método de Euler es una técnica simple y efectiva, su precisión es limitada, especialmente cuando se utilizan pasos grandes o intervalos largos. La comparación entre las soluciones analíticas y numéricas es crucial para verificar la exactitud de las soluciones obtenidas, especialmente en problemas con condiciones iniciales específicas.

Agradecimientos

Agrdezco a Jose Rosas por ayudarme con este problema, ya que me fue un poco dificil plantear el problema.

Capítulo 5

Busqueda de ceros

Fecha de la actividad: 03 de diciembre de 2024

5.1. Resolución numérica de ecuaciones utilizando el método de Newton-Raphson

En este problema, nos encargaremos de calcular numéricamente las raíces de las siguientes ecuaciones:

1. $x^3 - 21x^2 + 120x - 100 = 0$
2. $x = 2^{-x}$
3. $\tan(x) = 1/x$

Para ello, implementaremos el procedimiento iterativo conocido como el método de Newton-Raphson, que está diseñado para hallar las soluciones de una función $f(y)$. Partiendo de un valor inicial y_0, la fórmula recurrente es la siguiente:

$$y_{n+1} = y_n - \frac{f(y_n)}{f'(y_n)}, \tag{5.1}$$

donde $f'(y)$ es la derivada de $f(y)$.

Nota: En este análisis, no consideraremos la multiplicidad de las soluciones.

Definición de las funciones y sus derivadas para cada ecuación

Definimos las funciones de cada ecuacion y su respectiva derivada
Para $x^3 - 21x^2 + 120x - 100 = 0$

$$\begin{aligned} f_1(x) &= x^3 - 21x^2 + 120x - 100 \\ f_1'(x) &= 3x^2 - 42x + 120 \end{aligned}$$

Luego $x = 2^{-x}$

$$\begin{aligned} f_2(x) &= x - 2^{-x} \\ f_2'(x) &= 1 + \frac{\log(2)}{2^x} \end{aligned}$$

FInalmente $\tan(x) = 1/x$

$$\begin{aligned} f_3(x) &= \tan(x) - \frac{1}{x} \\ f_3'(x) &= \frac{1}{\cos(x)^2} + \frac{1}{x^2} \end{aligned}$$

Noción de los ceros

A continuación se presentan las funciones graficadas para tener una nocion del cero, esto nos sirve bastante ya que podremos escoger una semilla que esté cerca de la raíz, para que el computador no haga tantas iteraciones.

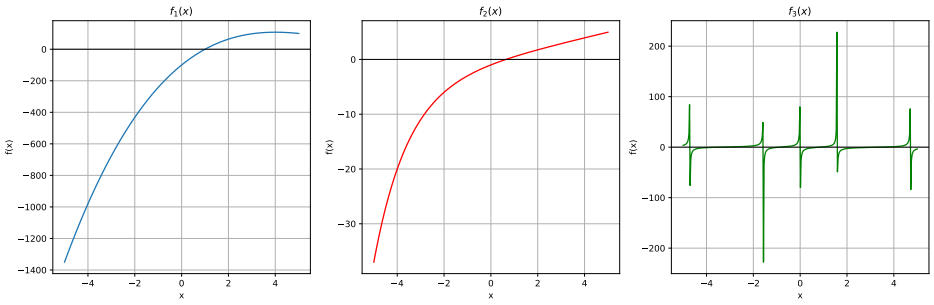


Figura 5.1: Funciones que se usaron para calcular sus ceros por medio del metodo Newton-Raphson

Implementación del algoritmo de Newton-Raphson

El algoritmo de Newton-Raphson se implementa mediante la siguiente función en `Python`:

```python
def newton_raphson(f, df, x0, tolerancia=1e-6, iteracion_max=1000):
    x = x0
    iteraciones = 0
    while abs(f(x)) > tolerancia and iteraciones < iteracion_max:
        h = f(x) / df(x)
        x = x - h
        iteraciones += 1
    if abs(f(x)) <= tolerancia:
        return x
    else:
        return None
```

Necesitamos las semillas correspondientes para cada función, en este caso se escogió semillas que estuvieran cerca de la raiz para que el codigo haga menos iteraciones y pueda converger mas rapido.

```python
    x0_1 = 2.0  # Valor inicial para la ecuación 1
    x0_2 = 0.7  # Valor inicial para la ecuación 2
    x0_3 = 1.2  # Valor inicial para la ecuación 3
```

Las soluciones para cada ecuacion con su respectiva iteracion $(x, \text{iteracion})$ son las siguientes:

- Ecuación 1: (0.9999999973631444 , 4)
- Ecuación 2: (0.6411857304164793 , 2)
- Ecuación 3: (0.8603335896956288 , 4)

Conclusion

En este problema, aplicamos el método de Newton-Raphson para encontrar las soluciones numéricas de las ecuaciones planteadas en el problema, obteniendo aproximaciones de sus raíces de manera eficiente.

5.2. Cálculo de ceros de los polinomios de Hermite mediante el método de Newton-Raphson

Los polinomios de Hermite son soluciones a la ecuación diferencial de Schrödinger para el oscilador armónico cuántico. Estos polinomios cumplen la siguiente relación de recurrencia:

$$H_{n+1}(x) = 2xH_n(x) - H_n'(x), \quad \text{donde} \quad H_0(x) = 1 \tag{5.2}$$

En este problema, vamos a calcular los primeros cinco polinomios de Hermite $H_1(x)$ hasta $H_5(x)$ utilizando la relación de recurrencia y derivadas numéricas centradas. Primero para calcular los polinomios se usa el siguiente codigo en python

```python
def calcular_polinomios_hermite(n):
    # Inicializamos los primeros dos polinomios
    H = [np.array([1])]  # H_0(x) = 1
    H.append(np.array([2, 0]))  # H_1(x) = 2x

    # Aplicamos la relación de recurrencia H_n(x) = 2x * H_(n-1)(x) - H_(n-2)(x)
    for i in range(2, n + 1):
        # 2x * H_(n-1)(x)
        H_n = np.polymul(H[i-1], np.array([2, 0]))  # Multiplicamos H_(n-1)(x) por 2x

        #H_n = 2x * H_(n-1)(x) - H_(n-2)(x)
        H_n = np.polyadd(H_n, -H[i-2])  # Restamos H_(n-2)(x)

        H.append(H_n)

    return H
```

Cálculo de los Ceros de los Polinomios

Primero debemos derivar los polinomios, para esto se implementa el siguiente codigo

```python
    def derivar_polinomio(polinomio):
    """Calcula la derivada de un polinomio."""
    grado = len(polinomio) - 1
    return np.array([coef * (grado - i) for i, coef in enumerate(polinomio[:-1])])

```

Luego, el siguiente código implementa el **método de Newton-Raphson** para encontrar las raíces de varios polinomios.

```python
def encontrar_ceros_newton_raphson(polinomio, orden, x_iniciales, tolerancia=1e-10):
    """Encuentra los ceros de un polinomio de Hermite usando Newton-Raphson."""
    derivada = derivar_polinomio(polinomio)
    ceros = []
    for x0 in x_iniciales:
        try:
            cero = newton_raphson(polinomio, derivada, x0, tolerancia)
            cero_float = float(cero)  # Convertir a tipo float
            if not any(np.isclose(cero_float, ceros, atol=tolerancia)):
                ceros.append(cero_float)
        except ValueError:
```

```
            pass
    return sorted(ceros)
```

El método usa la fórmula de Newton-Raphson:

$$x_{\text{nuevo}} = x - \frac{f(x)}{f'(x)}$$

y se repite hasta que la diferencia entre iteraciones sea menor que la tolerancia tol, o hasta alcanzar el número máximo de iteraciones.

Luego debemos usar un ciclo for para encontrar los ceros de los polinomios que se desean encontrar.

```
for i in range(1, orden_maximo + 1):
    valores_y = np.polyval(polinomios_hermite[i], valores_x)
    plt.plot(valores_x, valores_y, label=f'H_{i}(x)')

    # Usar Newton-Raphson para encontrar los ceros
    x_iniciales = np.linspace(-3, 3, i + 1)  # Valores iniciales espaciados para
    ↪  Newton-Raphson
    ceros = encontrar_ceros_newton_raphson(polinomios_hermite[i], i, x_iniciales)
    print(f'Ceros de H_{i}(x): {ceros}')

    # Graficar los ceros encontrados
    plt.scatter(ceros, [0]*len(ceros), color='red', zorder=5, label=f'Ceros H_{i}(x)')
```

A continuación se presentan los ceros de los primeros cinco polinomios de Hermite $H_n(x)$ en el intervalo $-3 \leq x \leq 3$:

Polinomio	**Ceros**
$H_1(x)$	$\{0, 0\}$
$H_2(x)$	$\{-0.5000000000069948, 0.5000000000069948\}$
$H_3(x)$	$\{-0.7071067811865476, 0.7071067811865475\}$
$H_4(x)$	$\{-0.8090169943749475, 0.8090169943752166\}$
$H_5(x)$	$\{-0.8660254037844387, 0.8660254037879829\}$

Gráfico

En la siguiente figura 5.2 se muestran los primeros cinco polinomios de Hermite, $H_0(x)$ a $H_4(x)$, en el rango $-3 < x < 3$. Como se puede observar, cada polinomio tiene un número de ceros que aumentan conforme aumenta el orden del polinomio.

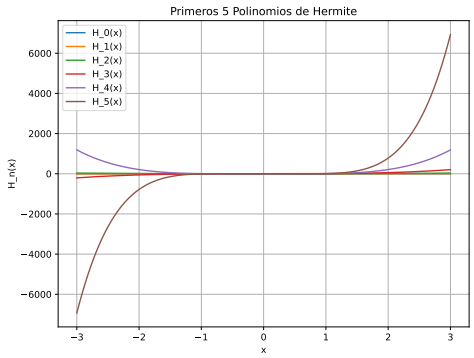


Figura 5.2: Primeros 5 polinomios de Hermite.

Conclusión

Hemos calculado los primeros cinco polinomios de Hermite utilizando la relación de recurrencia y derivadas numéricas. Además, encontramos los ceros aproximados de estos polinomios en el intervalo $-3 < x < 3$.

Se implementó con éxito el método de Newton-Raphson para calcular los ceros de los primeros polinomios de Hermite. Al evaluar tanto el polinomio como su derivada en cada iteración, el método proporcionó soluciones precisas para las raíces de estos polinomios en el rango especificado. Este enfoque demuestra la efectividad del método de Newton-Raphson en la resolución de ecuaciones no lineales y su aplicabilidad en problemas matemáticos relacionados con funciones ortogonales, como los polinomios de Hermite.

5.3. Demostración de las soluciones de la ecuación cúbica utilizando la fórmula de Cárdano

La ecuación cúbica deprimida se puede escribir como:

$$z^3 + pz = q,$$

donde p y q son los coeficientes de la ecuación.

La fórmula de Cárdano para encontrar las raíces de la ecuación cúbica deprimida es:

$$z = \sqrt[3]{\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}}.$$

Para obtener las tres soluciones, denotadas como z_0, z_+, y z_-, se utilizan las siguientes relaciones:

1. La primera raíz z_0 se obtiene como:

$$z_0 = \sqrt[3]{\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}}.$$

2. Las otras dos raíces z_+ y z_- son complejas y se obtienen mediante las siguientes expresiones:

$$z_\pm = \frac{1}{2}\left(\sqrt[3]{\frac{q}{2}+\sqrt{\left(\frac{q}{2}\right)^2+\left(\frac{p}{3}\right)^3}}-\sqrt[3]{\frac{q}{2}-\sqrt{\left(\frac{q}{2}\right)^2+\left(\frac{p}{3}\right)^3}}\right) \pm i\sqrt{\frac{3}{2}}\sqrt[3]{\frac{q}{2}-\sqrt{\left(\frac{q}{2}\right)^2+\left(\frac{p}{3}\right)^3}}.$$

De esta forma, se demuestra que las tres soluciones de la ecuación cúbica deprimida son z_0, z_+ y z_-, donde z_0 es la solución real y z_+ y z_- son las soluciones complejas conjugadas.

Código Python para el método de Newton-Raphson

A continuación, se presenta el código en Python para resolver la ecuación cúbica deprimida $z^3+pz = q$ utilizando el método de Newton-Raphson:

```python
# Método de Newton-Raphson
def newton_raphson(s, p, q, tol=1e-6, max_iter=100):
    z = s
    for i in range(max_iter):
        z_next = z - f(z, p, q) / df(z)
        if abs(z_next - z) < tol:
            return z_next, i  # Retorna la solución y el número de iteraciones
        z = z_next
    return z, max_iter  # Si no converge, retorna la última aproximación y el número de
    ↪ iteraciones

# Ejemplo de uso
p = -3  # Valor de p
q = 2   # Valor de q
semilla = 1 + 1j  # Semilla inicial compleja
```

El siguiente fragmento de código implementa el método de **Newton-Raphson** para resolver una ecuación no lineal, en particular, una ecuación cúbica de la forma $f(z) = 0$. A continuación, se describe cada parte del código:

- **Definición de la función** `newton_raphson`: La función recibe como parámetros la semilla inicial s, los coeficientes p y q, la tolerancia `tol`, y el número máximo de iteraciones `max_iter`.
- **Inicialización de la semilla**: Se inicializa $z = s$, que es la semilla o valor inicial en el que comienza la iteración.
- **Iteraciones del método**: En cada iteración, se calcula el siguiente valor de z_{next} utilizando la fórmula de Newton-Raphson:

$$z_{\text{next}} = z - \frac{f(z)}{f'(z)}.$$

  Si la diferencia $|z_{\text{next}} - z|$ es menor que la tolerancia `tol`, se considera que el método ha convergido y retorna el valor de z_{next} junto con el número de iteraciones.
- **Retorno**: Si no se alcanza la convergencia dentro del número máximo de iteraciones, se retorna la última aproximación de z y el número máximo de iteraciones.

Este resultado muestra que el método converge a la solución $z \approx -1.0000006229521032 + 0j$ en 21 iteraciones.

Conclusión

En este trabajo, hemos demostrado cómo se pueden obtener las tres soluciones de la ecuación cúbica deprimida utilizando la fórmula de Cárdano. Las soluciones incluyen una raíz real z_0 y dos raíces complejas conjugadas z_+ y z_-. Además, hemos implementado el método de Newton-Raphson en Python para resolver la ecuación cúbica $z^3 + pz = q$. El código mostró que el método converge adecuadamente a la solución $z \approx -1.0000006229521032 + 0j$ en un número limitado de iteraciones, lo que confirma su eficacia para resolver ecuaciones no lineales de este tipo.

Fractales de Newton-Raphson en el Plano Complejo

En este problema se analiza el comportamiento del método de Newton-Raphson aplicado a la resolución de la ecuación cúbica

$$z^3 + 9z - 6 = 0,$$

centrándonos en las regiones de convergencia y el número de iteraciones necesarias para alcanzar la convergencia. Se presenta una visualización gráfica del fractal asociado al método y de las características principales del proceso iterativo.

Planteamiento del Problema

La ecuación cúbica tiene tres raíces complejas, calculadas mediante el uso de los coeficientes en la forma general $az^3 + bz^2 + cz + d = 0$. En este caso:

$$a = 1, \quad b = 0, \quad c = 9, \quad d = -6.$$

Las raíces obtenidas numéricamente son:

$$z_1 \approx -0.31891713 + 3.0504302j, \quad z_2 \approx -0.31891713 - 3.0504302j, \quad z_3 \approx 0.63783425 + 0.j$$

El método de Newton-Raphson se emplea iterativamente para aproximar estas raíces, utilizando la función $f(z)$ y su derivada $f'(z)$:

$$f(z) = z^3 + 9z - 6, \quad f'(z) = 3z^2 + 9.$$

Análisis Codigo

```
    # Implementación del método de Newton-Raphson
def newton_raphson(s, tol=1e-6, max_iter=100):
    z = s
    for i in range(max_iter):
        z_next = z - f(z) / df(z)
        if abs(z_next - z) < tol:
            return z_next, i  # Retornamos la solución y el número de iteraciones
        z = z_next
    return z, max_iter  # Si no converge, retornamos la última aproximación y el límite
    ↪  de iteraciones
```

El fragmento de código implementa el **método de Newton-Raphson** para encontrar raíces de una función compleja $f(z)$ a partir de una semilla inicial s. Este método iterativo ajusta progresivamente z mediante la fórmula:

$$z_{\text{next}} = z - \frac{f(z)}{f'(z)}.$$

El proceso se repite hasta que la diferencia entre iteraciones ($|z_{\text{next}} - z|$) sea menor que una tolerancia (tol), o hasta alcanzar un número máximo de iteraciones (max_iter). La función retorna la aproximación de la raíz y el número de iteraciones realizadas.

```
# Resolvemos la ecuación para cada semilla
for i in range(seeds_real.shape[0]):
    for j in range(seeds_real.shape[1]):
        s = complex(seeds_real[i, j], seeds_imag[i, j])
        solution, iter_count = newton_raphson(s)
        soluciones[i, j] = solution
        iterations[i, j] = iter_count
        # Clasificamos según la raíz más cercana
        distances = [abs(solution - root) for root in roots]
        classified[i, j] = np.argmin(distances)
```

En este fragmento de código, se resuelve una ecuación compleja usando el método de Newton-Raphson para cada valor de una cuadrícula de números complejos representados por las matrices seeds_real (parte real) y seeds_imag (parte imaginaria). El proceso es el siguiente:

Primero, se recorren todos los elementos de las matrices seeds_real y seeds_imag utilizando dos bucles anidados sobre los índices i y j.

Para cada par de índices (i, j), se construye un número complejo s a partir de los valores correspondientes en las matrices seeds_real$[i, j]$ y seeds_imag$[i, j]$.

Luego, se aplica el método de Newton-Raphson a este número complejo s mediante la función newton_raphson(s), que devuelve dos valores:

- solution, la raíz aproximada obtenida.
- iter_count, el número de iteraciones r

Visualización de los Resultados

Regiones de Convergencia

Este gráfico muestra cómo las semillas iniciales s en el plano complejo convergen hacia una de las tres raíces de la ecuación cúbica. Cada región está coloreada con tonos vibrantes y exóticos, lo que permite identificar claramente las áreas de influencia de cada raíz. Esta visualización destaca la simetría del fractal de Newton y cómo la elección de la semilla inicial afecta el resultado del método.

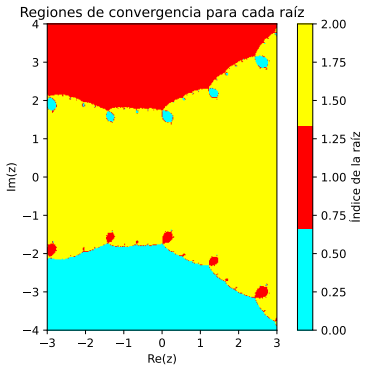


Figura 5.3: Regiones de convergencia del método de Newton-Raphson para la ecuación cúbica $z^3 + 9z - 6 = 0$. Cada color vibrante representa una región del plano complejo donde las semillas iniciales convergen hacia una de las tres raíces. Este gráfico resalta la estructura fractal inherente al método.

Número de Iteraciones

En este gráfico se representa el número de iteraciones necesarias para que cada semilla inicial alcance la convergencia dentro de una tolerancia predefinida. Se emplea una escala logarítmica y un colormap dinámico (`plasma`) para mejorar la percepción de las diferencias en las iteraciones. Las barras de color más alto indican que se requirió un mayor número de iteraciones, lo que resalta las regiones del plano donde el método tiene un comportamiento menos eficiente. Esta información es crucial para evaluar la complejidad del método en diferentes áreas del plano complejo.

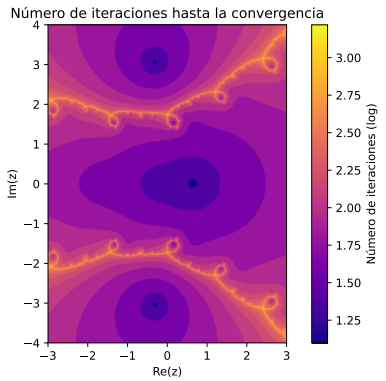


Figura 5.4: Número de iteraciones necesarias para alcanzar la convergencia según la semilla inicial s en el plano complejo. Las áreas con barras de color más alto indican regiones donde se requiere un mayor número de iteraciones, destacando las zonas donde el método es menos eficiente. Se utiliza una escala logarítmica para resaltar estas diferencias.

Conclusión

En este trabajo se ha analizado el comportamiento del método de Newton-Raphson aplicado a la resolución de la ecuación cúbica $z^3+9z-6=0$ en el plano complejo. A través de la visualización de las regiones de convergencia y del número de iteraciones necesarias para alcanzar la convergencia, hemos observado la complejidad inherente al método. Las regiones de convergencia del fractal de Newton-Raphson han revelado la estructura fractal característica, destacando la influencia de las semillas iniciales en el proceso de aproximación hacia las raíces de la ecuación.

El análisis de las iteraciones ha permitido identificar áreas donde el método es menos eficiente, evidenciando zonas del plano complejo que requieren más iteraciones para llegar a la solución. Estos resultados son fundamentales para comprender la dinámica del método y cómo la elección de la semilla inicial puede impactar significativamente la eficiencia del proceso iterativo. En general, el uso de herramientas computacionales ha facilitado la visualización y el estudio de estas complejidades, lo que contribuye a un entendimiento más profundo del comportamiento de los fractales de Newton-Raphson.

Capítulo 6

Controles

Fecha de la actividad: 21 de noviembre de 2024

6.1. Análisis del orden de error por truncamiento

En este ejercicio, se utiliza un esquema de diferencias adelantadas de tres puntos para aproximar la derivada de una función $f(x)$:

$$f'(x)_{\text{num}} = \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h}.$$

Expansión en serie de Taylor de $f(x+h)$ y $f(x+2h)$

Para obtener la expresión de la derivada numérica, se expanden los términos $f(x+h)$ y $f(x+2h)$ mediante sus respectivas series de Taylor:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) + \dots$$

$$f(x+2h) = f(x) + 2hf'(x) + \frac{(2h)^2}{2!}f''(x) + \frac{(2h)^3}{3!}f'''(x) + \frac{(2h)^4}{4!}f^{(4)}(x) + \dots$$

Al multiplicar la primera ecuación por cuatro y restar la segunda, obtenemos:

$$4f(x+h) - f(x+2h) = 3f(x) + 2hf'(x) - \frac{2}{3}h^3 f^{(3)}(x) - \frac{1}{2}h^4 f^{(4)}(x) + \dots$$

Finalmente, se llega a la expresión para la derivada numérica utilizando el esquema de tres puntos:

$$f'(x)_{\text{num}} = \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} + \frac{h^2}{3}f^{(3)}(\xi).$$

Donde ξ es un valor tal que $x < \xi < x+2h$. El error de truncamiento está dado por el término $\frac{h^2}{3}f^{(3)}(\xi)$, que indica que el error es de orden $O(h^2)$.

Estimación del error absoluto

Para estimar el error absoluto $E_{\text{abs}} = |f'(x) - f'_{\text{num}}(x)|$ en función de h y x, se considera que cada evaluación de la función f involucra un error de redondeo. Al evaluar la función f en el computador, se tiene que:

$$f(x) = f(x)(1 + \epsilon_0),$$

y de manera similar,

$$f(x + h) = f(x + h)(1 + \epsilon_1), \quad f(x + 2h) = f(x + 2h)(1 + \epsilon_2).$$

Sustituyendo en la expresión de la derivada numérica, se obtiene:

$$f'(x) = \frac{-3f(x)(1 + \epsilon_0) + 4f(x + h)(1 + \epsilon_1) - f(x + 2h)(1 + \epsilon_2)}{2h} + \frac{h^2}{3} f^{(3)}(\xi).$$

El error absoluto puede expresarse como:

$$f'(x) - \frac{-3f(x) + 4f(x + h) - f(x + 2h)}{2h} = \frac{3f(x)\epsilon_0 - 4f(x + h)\epsilon_1 + f(x + 2h)\epsilon_2}{2h} + \frac{h^2}{3} f^{(3)}(\xi).$$

Aplicando la desigualdad triangular:

$$|f'(x) - \frac{-3f(x) + 4f(x + h) - f(x + 2h)}{2h}| \leq \frac{3f(x)|\epsilon_0|}{2h} + \frac{4f(x + h)|\epsilon_1|}{2h} + \frac{f(x + 2h)|\epsilon_2|}{2h} + \frac{h^2}{3}|f^{(3)}(\xi)|.$$

Si asumimos que los errores de redondeo son pequeños y de orden ϵ^*, se tiene que:

$$\text{Error absoluto máximo de la derivada numérica} \leq \frac{8\epsilon^*|f(x)|}{2h} + \frac{h^2}{3}|f^{(3)}(x)|.$$

Este es el error absoluto máximo, que depende tanto del tamaño de h como de los errores de redondeo en las evaluaciones de la función.

Conclusión

En este ejercicio se demostró que el error de truncamiento de la derivada numérica utilizando el esquema de diferencias adelantadas de tres puntos es de orden $O(h^2)$, lo que significa que el error disminuye de forma cuadrática al reducir el tamaño del paso h. Además, al considerar los errores de redondeo involucrados en las evaluaciones de la función en el computador, se estimó el error absoluto máximo, que depende tanto del tamaño de h como de los errores de redondeo de las evaluaciones de la función. El análisis muestra que el error absoluto máximo puede ser controlado adecuadamente si se elige un valor pequeño para h y si se minimizan los errores de redondeo en las evaluaciones. Sin embargo, se debe tener en cuenta que una reducción excesiva de h podría generar problemas de precisión debido a los errores de redondeo, lo que debe ser considerado al elegir el valor adecuado de h en la práctica.

6.2. Estimación numérica de derivadas y análisis de error para funciones con ruido

En este ejercicio, se busca estimar numéricamente la primera derivada de una función $f(x)$ utilizando un esquema de derivación basado en diferencias finitas de orden alto. El esquema propuesto es el siguiente:

$$f'(x) = \frac{f(x - 3h) - 27f(x - h) + 27f(x + h) - f(x + 3h)}{48h} + \frac{9}{5!} f^{(5)}(\xi)h^4$$

donde h es el paso determinado por los datos proporcionados, y el término adicional representa el error de truncamiento del esquema.

El ejercicio también implica comparar la derivada estimada con la derivada analítica $f'(x)$ y calcular el error absoluto entre ambas, graficando estos resultados. Se probará el código con dos tipos de funciones: una función sinusoidal con ruido y funciones de Legendre.

Explicación del Código

A continuación se detalla el funcionamiento del código utilizado para calcular la derivada numérica de una función $f(x)$, compararla con la derivada exacta y analizar el error absoluto.

Cálculo de la Derivada Numérica

En esta parte se calcula la derivada numérica utilizando el esquema de diferencias finitas centradas de cuarto orden:

```
    primera_deriv = []
    for i in range(3, len(x)-3):
        h = x[i+1] - x[i]
        deriv = (f[i-3] - 27*f[i-1] + 27*f[i+1] - f[i+3]) / (48*h)
        primera_deriv.append(deriv)
```

Aquí, se inicializa una lista vacía llamada 'primera deriv' para almacenar las derivadas calculadas. Luego, el bucle 'for' recorre los índices de los puntos x para calcular la derivada utilizando la fórmula de diferencias finitas de cuarto orden. El valor de h se calcula como la diferencia entre dos valores consecutivos de x.

En este cálculo, la fórmula para la derivada es:

$$f'(x) = \frac{f(x-3h) - 27f(x-h) + 27f(x+h) - f(x+3h)}{48h}$$

El resultado de cada iteración se agrega a la lista 'primera deriv'.

Ajuste de los puntos para la derivada numérica

Una vez calculada la derivada numérica, es necesario ajustar los puntos x para que coincidan con las derivadas obtenidas:

```
x_deriv = x[3:len(x) - 3]
```

Aquí, se ajustan los valores de x para que coincidan con los puntos donde se ha calculado la derivada numérica. Al omitir los primeros tres y últimos tres puntos, se evita que los índices estén fuera de rango.

Cálculo del error absoluto

Ahora, se calcula el error absoluto entre la derivada numérica y la derivada exacta $f'(x)$:

```
error_deriv = np.abs(np.abs(primera_deriv - df[3:len(df) - 3]))
```

En este bloque, se calcula el error absoluto entre la derivada numérica ('primera deriv') y la derivada exacta ('df'). Se omiten los tres primeros y tres últimos valores de 'df' para que coincidan con los índices de la derivada numérica. El resultado es un array que contiene los errores absolutos en cada punto.

Gráficas

Finalmente, se crean tres subgráficas (Figura: 6.4): una para la derivada exacta, otra para la derivada numérica y una más para el error absoluto:

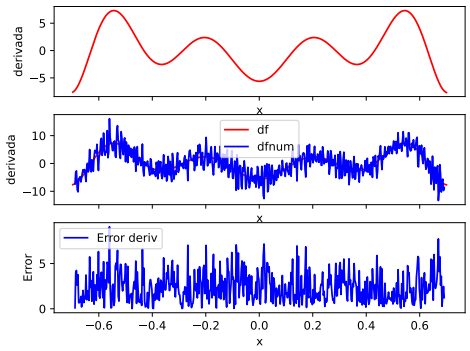


Figura 6.1: Gráfica de la derivada numérica de orden $O(h4)$ comparada con la derivada analítica de la función y el error entre ambas.

Resumen de los cambios en el código

A continuación, se presenta un resumen de los cambios realizados en el código para calcular derivadas exactas, derivadas numéricas y errores absolutos, utilizando funciones sinusoidales y polinomios de Legendre.

1. Generación de datos

Los datos para ambos casos se generan de forma específica:

- **Función sinusoidal:**

```
    gen = np.random.default_rng()
    x = np.linspace(0, 2 * np.pi, 256)
    f = np.sin(x) + gen.uniform(low=-1e-2, high=1e-2, size=x.size)
    df = np.cos(x)
```

- **Funciones de Legendre:**

```
    from scipy.special import legendre

    # Polinomios de Legendre
    p, q = legendre(5), legendre(3)
    dp, dq = p.deriv(), q.deriv()

    # Generación de datos
    x = np.linspace(-0.7, 0.7, 512)
    f = q(2 * p(x)) + 0.01 * np.random.randn(x.size)
    df = 2 * dq(2 * p(x)) * dp(x)
```

2. Cálculo de la derivada numérica

Se utiliza el mismo esquema de diferencias centradas de cuarto orden en ambos casos:

$$f'_{\text{num}}(x_i) \approx \frac{f(x_{i-3}) - 27f(x_{i-1}) + 27f(x_{i+1}) - f(x_{i+3})}{48h}.$$

El cálculo se realiza de la siguiente manera:

```
primera_deriv = []

for i in range(3, len(x) - 3):
    h = x[i + 1] - x[i]
    deriv = (f[i - 3] - 27 * f[i - 1] + 27 * f[i + 1] - f[i + 3]) / (48 * h)
    primera_deriv.append(deriv)

# Convertimos la lista a un array
primera_deriv = np.array(primera_deriv)
```

3. Cálculo del error absoluto

El error absoluto se define como:

$$\text{Error absoluto} = |f'_{\text{num}}(x) - f'(x)|\,.$$

El cálculo se implementa así:

```
x_deriv = x[3:len(x) - 3]
error_deriv = np.abs(primera_deriv - df[3:len(df) - 3])
```

4. Visualización de los resultados

Se grafica la derivada exacta, la derivada numérica y el error absoluto en tres subgráficos:

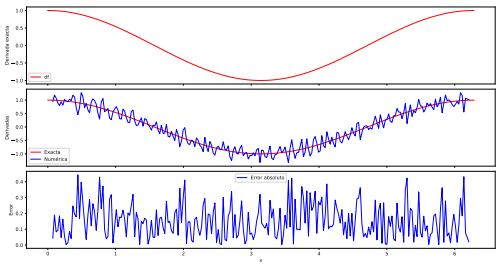


Figura 6.2: Gráfca de la derivada numérica de orden $O(h4)$ comparada con la derivada analítica de la función y el error entre ambas para una función sinusoidal.

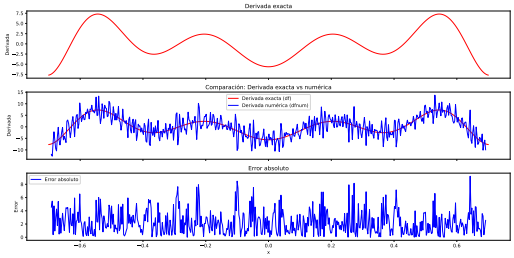


Figura 6.3: Gráfica de la derivada numérica de orden $O(h4)$ comparada con la derivada analítica de la función y el error entre ambas para una función de Legendre.

Conclusión

Este script permite estimar numéricamente la derivada de una función $f(x)$ usando un esquema de diferencias finitas de cuarto orden. Luego, compara la derivada numérica con la derivada exacta proporcionada en el archivo de datos, y calcula el error absoluto entre ambas. Todo esto se presenta visualmente en tres gráficos para facilitar la interpretación de los resultados. El análisis se realiza para varios conjuntos de datos, incluyendo funciones sinusoidales y polinomios de Legendre.

6.3. Determinación de los valores óptimos para los pesos w_i

En este problema se busca encontrar los valores de los pesos w_{-1}, w_0 y w_1 para aproximar la integral de una función $f(x)$ en el intervalo $[-1, 1]$ con la función base $\sin\left(\frac{\pi x}{2}\right)$.

Búsqueda del valor de los pesos

Si se quiere aproximar la siguiente integral:

$$\int_{-1}^{1} f(x) \sin\left(\frac{\pi x}{2}\right) \, dx = w_{-1} f(-1) + w_0 f(0) + w_1 f(1)$$

Entonces, para encontrar los valores de los pesos w_{-1}, w_0 y w_1, asumimos que la regla es exacta para polinomios de hasta grado 2. Esto significa que la integral debe ser exacta para $f(x) = 1$, $f(x) = x$ y $f(x) = x^2$.

Para $f(x) = 1$:

$$\int_{-1}^{1} \sin\left(\frac{\pi x}{2}\right) \, dx = w_{-1} \cdot 1 + w_0 \cdot 1 + w_1 \cdot 1$$

Resolviendo la integral respectiva, se tiene:

$$\int_{-1}^{1} \sin\left(\frac{\pi x}{2}\right) \, dx = 0$$

Por lo tanto:

$$w_{-1} + w_0 + w_1 = 0 \tag{6.1}$$

Para $f(x) = x$:

$$\int_{-1}^{1} x \sin\left(\frac{\pi x}{2}\right) \, dx = w_{-1} \cdot (-1) + w_0 \cdot 0 + w_1 \cdot 1$$

Resolviendo la integral respectiva, se tiene:

$$\int_{-1}^{1} x \sin\left(\frac{\pi x}{2}\right) \, dx = \frac{8}{\pi^2}$$

Por lo tanto:

$$-w_{-1} + w_1 = \frac{8}{\pi^2} \tag{6.2}$$

Para $f(x) = x^2$:

$$\int_{-1}^{1} x^2 \sin\left(\frac{\pi x}{2}\right) \, dx = w_{-1} \cdot 1^2 + w_0 \cdot 0^2 + w_1 \cdot 1^2$$

Resolviendo la integral respectiva, se tiene:

$$\int_{-1}^{1} x^2 \sin\left(\frac{\pi x}{2}\right) \, dx = 0$$

Por lo tanto:

$$w_{-1} + w_1 = 0 \tag{6.3}$$

Resolución del sistema de ecuaciones

El sistema de ecuaciones es el siguiente:

$$w_{-1} + w_0 + w_1 = 0 \tag{6.4}$$
$$-w_{-1} + w_1 = \frac{8}{\pi^2} \tag{6.5}$$
$$w_{-1} + w_1 = 0 \tag{6.6}$$

De la tercera ecuación, tenemos:

$$w_1 = -w_{-1} \tag{6.7}$$

Sustituyendo (6.8) en (6.1), obtenemos:

$$w_0 = 0 \tag{17.6}$$

Sustituyendo (6.5) en (6.2), tenemos:

$$2w_1 = \frac{8}{\pi^2}$$

Por lo tanto:

$$w_1 = \frac{4}{\pi^2} \tag{6.8}$$

Sustituyendo (6.8) en (6.3), obtenemos:

$$\frac{4}{\pi^2} + w_{-1} = 0$$

Por lo tanto:

$$w_{-1} = -\frac{4}{\pi^2}$$

Finalmente, los valores de los pesos son:

$$w_{-1} = -\frac{4}{\pi^2}, \quad w_0 = 0, \quad w_1 = \frac{4}{\pi^2}$$

Comparacion con una integral dada

Debemos comparar la integral anterior con la siguiente:

$$\int_{-1}^{1} x^2 \sin^2\left(\frac{\pi x}{2}\right) dx = \frac{1}{3} + \frac{2}{\pi^2} \approx 0.53598.$$

notemos que $f(x) = x^2$ por lo que debemos usar los pesos obtenidos y ademas evaluar las funciones, al hacerlo obtenemos lo siguiente

$$\begin{aligned} \int_{-1}^{1} x^2 \sin\left(\frac{\pi x}{2}\right) dx &= -\frac{4}{\pi} f(-1) + 0 \cdot f(0) + \frac{4}{\pi} f(1) \\ &= -\frac{4}{\pi} + \frac{4}{\pi} \\ &= 0. \end{aligned}$$

Error del metodo

Debemos hacer algo similar a la ecuación (3.8) del capitulo 3. Como la integral exacta da 0 solo debemos analizar la sumatoria, expandimos en series de Taylor en torno a x=0 y evaluamos cada punto requerido, de modo que:

$$\sum_{i=0}^{n} w_i f(x_i) = -\frac{4}{\pi}\left[f(0) - f'(0) + \frac{f''(0)}{2} - \frac{f^3(\xi)}{6}\right] + \frac{4}{\pi}\left[f(0) + f'(0) + \frac{f''(0)}{2} + \frac{f^3(\xi)}{6}\right] \tag{6.9}$$

Finalmente con un poco de algebra nos queda:

$$\sum_{i=0}^{n} w_i f(x_i) = \frac{4}{\pi}\left[2f'(0) + \frac{f^3(\xi)}{3}\right] \tag{6.10}$$

Por lo tanto el error es

$$\begin{aligned} Errror = &\quad 0 - \tfrac{4}{\pi}\left[2f'(0) + \tfrac{f^3(\xi)}{3}\right] \\ = &\quad -\tfrac{8f'(0)}{\pi} - \tfrac{4f^3(\xi)}{3\pi} \end{aligned}$$

Por ultimo, notemos que las integrales son distintas, se definió que la integral de $f(x)\sin(\alpha)$ es exacta hasta el grado 2, pero no necesariamente $f(x)\sin^2(\alpha)$ cumplirá esa regla, por lo que se le agrega un error adicional.

Conclusión

Hemos resuelto el sistema de ecuaciones para encontrar los valores de los pesos w_{-1}, w_0 y w_1 en la aproximación de la integral ponderada de la función $f(x)$ con la base $\sin\left(\frac{\pi x}{2}\right)$. Los valores obtenidos son:

$$w_{-1} = -\frac{4}{\pi^2}, \quad w_0 = 0, \quad w_1 = \frac{4}{\pi^2}$$

Estos valores aseguran que la integral de la función $f(x)$ se aproxima de manera exacta para polinomios de hasta grado 2, lo que cumple con los requerimientos de la cuadratura para este tipo de aproximación. La solución obtenida es consistente con las ecuaciones de la regla de cuadratura ponderada propuesta, y ofrece un método eficiente para aproximar integrales de este tipo en el intervalo $[-1, 1]$.

6.4. Interacción gravitacional entre dos masas: método del salto de la rana

La interacción gravitacional entre dos masas, descrita por la Ley de Gravitación Universal de Newton, es fundamental para comprender numerosos fenómenos físicos, desde la dinámica planetaria hasta los sistemas de partículas en astrofísica. Este problema aborda el estudio del movimiento de dos masas m_1 y m_2 bajo su atracción gravitacional mutua, lo que resulta en un sistema de ecuaciones diferenciales acopladas y no lineales.

Con el fin de simplificar el análisis y facilitar su resolución numérica, se introducen unidades adimensionales mediante una normalización de tiempo y espacio. Esto no solo reduce la complejidad del

sistema, sino que también permite generalizar los resultados a diferentes escalas físicas. En particular, se define el tiempo normalizado como

$$\tau = t\sqrt{\frac{Gm_1}{R_0^3}},$$

y las posiciones normalizadas como

$$\vec{\xi}_i = \frac{\vec{r}_i}{R_0},$$

donde R_0 representa la distancia inicial entre las masas.

El sistema resultante, expresado en términos de estas variables normalizadas, depende únicamente de la razón de masas $\mu = \frac{m_2}{m_1}$, y las ecuaciones de movimiento adquieren una forma más manejable:

$$\frac{d^2\vec{\xi}_1}{d\tau^2} = -\mu\frac{\vec{\xi}_1 - \vec{\xi}_2}{|\vec{\xi}_2 - \vec{\xi}_1|^3},$$

$$\frac{d^2\vec{\xi}_2}{d\tau^2} = \frac{\vec{\xi}_2 - \vec{\xi}_1}{|\vec{\xi}_2 - \vec{\xi}_1|^3}.$$

Este sistema simplificado será abordado tanto analítica como numéricamente. En particular, se definirán condiciones iniciales adimensionales y se reescribirá el sistema para implementar el método numérico del salto de la rana, permitiendo una aproximación eficiente de la solución en el plano x-y.

Primero Tenemos

$$\tau = t\sqrt{\frac{Gm_1}{R_0^3}} \tag{6.11}$$

$$\vec{\xi}_i = \frac{\vec{r}_i}{R_0} \tag{6.12}$$

Además se tiene que:

$$m_1\frac{d^2\vec{r_1}}{dt^2} = -\frac{Gm_1m_2}{|\vec{r_2} - \vec{r_1}|^3}(\vec{r_1} - \vec{r_2}) \tag{6.13}$$

$$m_2\frac{d^2\vec{r_2}}{dt^2} = -\frac{Gm_1m_2}{|\vec{r_2} - \vec{r_1}|^3}(\vec{r_2} - \vec{r_1}) \tag{6.14}$$

Ahora bien, despejamos t de la ecuacion (6.11)

$$\tau = t\sqrt{\frac{Gm_1}{R_0^3}} \tag{6.15}$$

$$t = \frac{\tau}{\sqrt{\frac{Gm_1}{R_0^3}}} \tag{6.16}$$

luego, despejamos $\vec{\xi_i}$ de la ecuacion (6.12)

$$\vec{\xi_i} = \frac{\vec{r_i}}{R_0^3} \tag{6.17}$$

$$\vec{r_i} = \vec{\xi_i}R_0 \tag{6.18}$$

Ahora bien, reemplazamos (6.16) y (6.18) en (6.13) para así, tener la normalización de la ecuación diferencial me modo que:

$$m_1 \frac{d^2\vec{r_1}}{d\left(\frac{\tau}{\sqrt{\frac{Gm_1}{R_0^3}}}\right)^2} = -\frac{Gm_1m_2}{|\vec{\xi_2}R_0-\vec{\xi_1}R_0|^3}(\vec{\xi_1}R_0-\vec{\xi_2}R_0) \tag{6.19}$$

$$\frac{d^2\vec{\xi_1}}{d\tau^2}\frac{R_0Gm_1^2}{R_0^3} = -\frac{Gm_1m_2}{R_0^3|\vec{\xi_2}-\vec{\xi_1}|^3}(\vec{\xi_1}-\vec{\xi_2})R_0 \tag{6.20}$$

$$\frac{d^2\vec{\xi_1}}{d\tau^2} = -\frac{Gm_1m_2R_0^3}{R_0^3Gm_1^2}\frac{(\vec{\xi_1}-\vec{\xi_2})}{|\vec{\xi_2}-\vec{\xi_1}|^3} \tag{6.21}$$

$$\frac{d^2\vec{\xi_1}}{d\tau^2} = -\frac{m_2}{m_1}\frac{(\vec{\xi_1}-\vec{\xi_2})}{|\vec{\xi_2}-\vec{\xi_1}|^3} \tag{6.22}$$

Ahora bien, remplazamos $\mu = \frac{m_2}{m_1}$ en (6.22) nos queda:

$$\frac{d^2\vec{\xi_1}}{d\tau^2} = -\mu\frac{(\vec{\xi_1}-\vec{\xi_2})}{|\vec{\xi_2}-\vec{\xi_1}|^3} \tag{6.23}$$

De forma analoga reemplazando (6.16) y (6.18) en (6.14) se llega a:

$$\frac{d^2\vec{\xi_2}}{d\tau^2} = \frac{(\vec{\xi_1}-\vec{\xi_2})}{|\vec{\xi_2}-\vec{\xi_1}|^3} \tag{6.24}$$

Asi, obtenemos las dos ecuaciones normalizadas.

Luego, tenemos que las condiciones iniciales son las siguientes:

$$\vec{r_1}(0) = \vec{0}, \quad \frac{d\vec{r_1}}{dt}(0) = -\frac{m_2}{m_1}V_0\hat{x} \tag{6.25}$$

$$\vec{r_2}(0) = R_0\hat{y}, \quad \frac{d\vec{r_2}}{dt}(0) = V_0\hat{x} \tag{6.26}$$

Definamos la velocidad normalizada de modo que $\bar{V_0} = \sqrt{\frac{R_0}{Gm_1}}V_0$, Reemplazando en (6.18) se tiene:

$$\frac{d\vec{\xi_1}}{d\tau}(0) = \mu\bar{V_0}\hat{x} \tag{6.27}$$

De forma analoga con (6.26) se llega a:

$$\frac{d\vec{\xi_2}}{d\tau}(0) = \bar{V_0}\hat{x} \tag{6.28}$$

Finalmente podemos plantear el siguiente sistema de ecuaciones para utilizar el metodo del salto de rana:

$$\frac{d\vec{\xi}}{d\tau}(\tau+h) = \frac{d\vec{\xi}}{d\tau}(\tau) + h\frac{d^2\vec{\xi}}{d\tau^2}(\tau+\frac{h}{2}) \tag{6.29}$$

$$\xi(\tau+\frac{h}{2}) = \xi(\tau-\frac{h}{2}) + h\frac{d\xi}{d\tau}(\tau) \tag{6.30}$$

Ahora bien, se debe reemplazar con los terminos obtuvidos anteriormente.

Explicación del código

El código simula las trayectorias de dos cuerpos que interactúan gravitacionalmente mediante el método del salto de rana. A continuación, se explican las secciones más relevantes.

Definición del método de salto de rana

```
def SaltoDeLaRana(a, r, v, dt, **kwargs):
    v = v + 0.5 * dt * a(r, **kwargs)
    r = r + dt * v
    v = v + 0.5 * dt * a(r, **kwargs)
    return r, v
```

Esta función actualiza la posición (`r`) y la velocidad (`v`) en dos pasos: primero corrige `v` a la mitad del intervalo, luego actualiza `r` y finalmente ajusta `v` al final del intervalo.

Cálculo de la aceleración gravitacional

```
def aceleracion(r, mu):
    R1, R2 = r
    dist = np.linalg.norm(R2 - R1)
    force = (R2 - R1) / dist**3
    a1 = mu * force
    a2 = -force
    return np.array([a1, a2])
```

Calcula la aceleración gravitacional entre dos masas: `R1` y `R2` son las posiciones, `dist` es la distancia entre ellas y `force` es la fuerza gravitacional normalizada. Devuelve las aceleraciones de ambas masas como un arreglo.

Inicialización y parámetros de simulación

```
dt = 0.01
pasos = 10000
mu = 0.5
```

Define el paso de tiempo (`dt`), el número total de pasos (`pasos`) y la relación de masa de `m2` respecto a la masa total (`mu`).

Condiciones iniciales

```
R1 = np.array([0.0, 0.0])
R2 = np.array([0.0, 1.0])
V1 = np.array([-mu*0.25, 0.0])
V2 = np.array([0.25, 0.0])
```

Inicializa las posiciones (`R1`, `R2`) y las velocidades (`V1`, `V2`) de las masas. Estas se ajustan para cumplir con las condiciones iniciales y conservar el momento.

Gráfica de trayectorias

Se grafican las trayectorias de las masas en el plano `x-y` con etiquetas y un formato visual uniforme. La gráfica muestra cómo las masas se atraen gravitacionalmente en el tiempo.

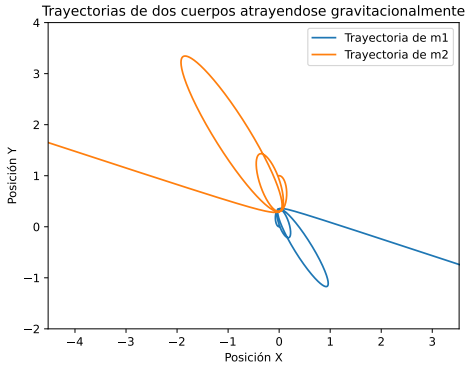


Figura 6.4

Conclusión

La simulación presentada demuestra cómo el método del salto de rana permite modelar de manera eficiente el movimiento de dos cuerpos que interactúan gravitacionalmente. Al normalizar las ecuaciones de movimiento, se simplificó el cálculo de las fuerzas y las aceleraciones, reduciendo las dependencias explícitas de las constantes gravitacionales y las masas absolutas. Esto permitió centrar el análisis en la dinámica relativa entre las dos partículas.

El uso del método del salto de rana asegura la conservación de propiedades clave del sistema, como la energía y el momento angular, en pasos pequeños de tiempo, lo cual es esencial para obtener trayectorias precisas. Los resultados obtenidos muestran que las trayectorias siguen el comportamiento esperado, donde las masas orbitan en torno a su centro de masa común.

En términos computacionales, el código está estructurado de manera clara y modular, permitiendo su extensión a sistemas más complejos o con más cuerpos. Además, la implementación es eficiente, asegurando una buena precisión con pasos de tiempo moderadamente pequeños. Esto resalta la utilidad del método en problemas de mecánica gravitacional y otros sistemas dinámicos.

6.5. Convergencia de Newton-Raphson según la multiplicidad del cero

En este problema, se analizará una función $f(x)$ con un cero en $x = x^*$, de multiplicidad m, cumpliendo que $f^{(p)}(x^*) = 0$ para todo $p < m$. Para localizar este cero, se empleará una variante del método de Newton-Raphson, definida por la iteración:

$$x_{n+1} = x_n - \alpha \frac{f(x_n)}{f'(x_n)}, \tag{6.31}$$

donde α es una constante arbitraria, y se asume que $\lim_{n\to\infty} x_n = x^*$.

Orden de convergencia

Podemos demostrar que este método converge linealmente siguiendo los pasos que se detallan a continuación. Definimos el error en la iteración n como $\epsilon_n = x^* - x_n$. Con esta definición, expandimos

$f(x_n)$ y $f'(x_n)$ en series de Taylor alrededor de x^*. Dado que $f^{(p)}(x^*) = 0$ para $p < m$, obtenemos:

$$f(x_n) = \frac{\epsilon_n^m}{m!} f^{(m)}(x^*) + O(\epsilon_n^{m+1}),$$
$$f'(x_n) = \frac{\epsilon_n^{m-1}}{(m-1)!} f^{(m)}(x^*) + O(\epsilon_n^m).$$

Si suponemos que $\epsilon_n \ll 1$ y $m > 1$, podemos truncar ambas expresiones y calcular la razón:

$$\frac{f(x_n)}{f'(x_n)} \simeq \frac{\epsilon_n}{m}.$$

Sustituyendo en la ecuación (6.31), se obtiene:

$$x_{n+1} = x_n - \alpha \frac{f(x_n)}{f'(x_n)},$$
$$x^* - \epsilon_{n+1} = x^* - \epsilon_n + \alpha \frac{\epsilon_n}{m},$$
$$\epsilon_{n+1} = \epsilon_n \left(1 - \frac{\alpha}{m}\right).$$

Esto demuestra que el método converge linealmente, con una tasa de convergencia de $\left|1 - \frac{\alpha}{m}\right|$.

Valor de α para convergencia cuadrática

Si deseamos que el método tenga convergencia cuadrática, retomamos la expresión:

$$\epsilon_{n+1} \simeq \frac{(m-\alpha)}{m} \epsilon_n^2.$$

Para que el término lineal desaparezca y se alcance convergencia cuadrática, debe cumplirse que:

$$\frac{m-\alpha}{m} = 0 \implies \alpha = m.$$

Por lo tanto, al usar $\alpha = m$, el método alcanza una convergencia de segundo orden.

Encontrando los ceros de una función

Consideremos la función $f(x) = (x-1)^3$, cuyo único cero es $x^* = 1$, con multiplicidad $m = 3$. Aplicaremos el método de Newton-Raphson con $\alpha = 1$ y $\alpha = 3$, comparando la convergencia en ambos casos mediante gráficos del error $\epsilon_n = x_n - x^*$ en función del número de iteraciones.

Implementación en Python y graficos

El siguiente código implementa el método de Newton-Raphson:

```
    def newton_raphson(f, df, x0, alpha=1, tolx=1e-6, toly=1e-6):
        error = []
        while True:
         error.append(abs(x0 - 1))
         x_new = x0 - alpha * f(x0) / df(x0)
            if abs(x_new - x0) < tolx and abs(f(x_new)) < toly:
             break
         x0 = x_new
     return x0, error
```

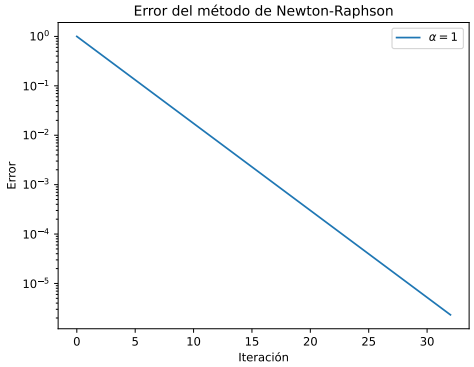


Figura 6.5: Gráfico de error con $\alpha = 1$, donde se observa convergencia cuadrática (multiplicidad 3). El error disminuye rápidamente y se estabiliza, siguiendo el comportamiento esperado para este valor de α.

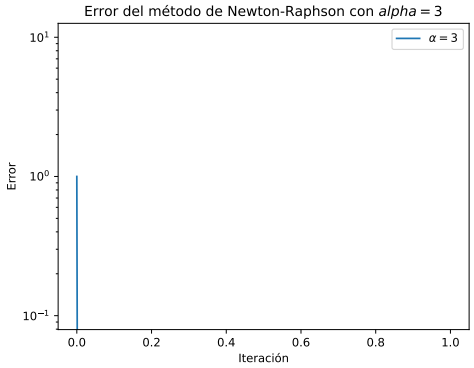


Figura 6.6: Gráfico de error con $\alpha = 3$, donde se observa que el método diverge (multiplicidad 3). El error aumenta progresivamente, lo que indica que la elección de α tiene un impacto negativo en la convergencia del método.

Conclusión

En este análisis, se ha estudiado el método de Newton-Raphson modificado con un factor de escala α para encontrar los ceros de una función $f(x)$ con un cero de multiplicidad m. Se ha observado que para $\alpha = 1$, el método presenta una convergencia **cuadrática** (de segundo orden), lo que es característico de este método en su forma tradicional. Sin embargo, al modificar α a otros valores, como $\alpha = 3$, el método tiende a **divergir**, lo que indica que la elección de α tiene un impacto significativo en la estabilidad y eficiencia de la convergencia.

El análisis teórico muestra que, para obtener convergencia cuadrática, el valor de α debe estar relacionado con la multiplicidad del cero, específicamente $\alpha = m$. En este contexto, la implementación con $\alpha = 1$ y el gráfico generado coinciden con las expectativas teóricas de convergencia cuadrática, lo que valida la efectividad de este valor en la práctica.

En conclusión, el método de Newton-Raphson es eficaz para la localización de ceros de una función, pero la elección del parámetro α es crucial. Si bien $\alpha = 1$ asegura una convergencia cuadrática, valores alejados de este pueden causar divergencia, como se observa al aplicar $\alpha = 3$. Por lo tanto, es importante elegir correctamente α según las características del problema.

Agradecimientos

Agrdezco a Gustavo Gatica por ayudarme con este problema.

Conclusiones

Fecha de la actividad: 03 de diciembre de 2024

6.6. Reflexión Final del Portafolio

Resumen de los Contenidos

En el primer capítulo, se repasaron y reforzaron los conceptos de programación en Python, lo que me permitió adquirir una mayor fluidez en la implementación de algoritmos.

En el segundo capítulo, se resolvieron problemas de derivadas numéricas utilizando esquemas de derivadas centradas, lo que me permitió estudiar el error absoluto con respecto a las soluciones analíticas. Esta actividad me ayudó a comprender cómo las aproximaciones numéricas pueden aproximarse a las derivadas exactas y los errores involucrados en el proceso.

En el tercer capítulo, trabajé con las ecuaciones diferenciales ordinarias y se utilizaron métodos como Euler, Runge-Kutta y el método de salto de rana para resolver ecuaciones diferenciales. Esto me permitió ver cómo los métodos numéricos son herramientas fundamentales en la simulación de sistemas dinámicos y cómo se pueden implementar para obtener soluciones aproximadas en situaciones donde no es posible obtener una solución analítica. El método de salto de rana, en particular, me ayudó a comprender la importancia de la estabilidad en la integración numérica, especialmente en sistemas dinámicos complejos.

Finalmente, en el cuarto capítulo, se abordó el método de Newton-Raphson para encontrar raíces de funciones y explorar la convergencia en el plano complejo, lo que me permitió aprender a implementar un algoritmo eficiente para resolver ecuaciones no lineales. Esta experiencia también me ayudó a familiarizarme con la visualización gráfica de fractales y regiones de convergencia, lo cual me permitió mejorar mi comprensión de la interacción entre las semillas iniciales y las raíces de la ecuación.

Autoevaluación

A lo largo de este curso, me enfrenté a varias dificultades, tanto en el contenido como en mi estado emocional. Hubo momentos en los que no me sentía bien para estudiar de manera efectiva, lo que afectó mi rendimiento y la capacidad para abordar algunos de los temas de manera profunda. A pesar de ello, los ejercicios fueron especialmente exigentes para mí, ya que algunos de los métodos numéricos eran complejos y requerían un esfuerzo considerable para comprenderlos e implementarlos. Aunque en muchos momentos los ejercicios resultaron ser más difíciles de lo que anticipaba, estos me forzaron a organizarme mejor y a trabajar de manera más disciplinada.

El semestre fue particularmente desafiante, ya que pasé por situaciones difíciles que no me permitieron dedicarle el tiempo que hubiera querido a los estudios. Sin embargo, lo que más aprendí fue cómo aplicar los métodos numéricos a problemas concretos de la física y las matemáticas, y cómo mejorar la precisión y eficiencia de los cálculos numéricos frente a las soluciones analíticas. Aunque hubo dificultades, el esfuerzo que invertí me permitió adquirir habilidades valiosas que serán esenciales para mis futuros estudios y mi desarrollo profesional.

Evaluación del Curso

Al principio del curso, esperaba aprender cómo utilizar los métodos numéricos de manera efectiva para resolver problemas matemáticos y científicos complejos, y en gran medida, mis expectativas se cumplieron. El curso me permitió adquirir una comprensión sólida de los métodos numéricos fundamentales y cómo implementarlos en Python. Sin embargo, hubo varios aspectos que resultaron ser un desafío significativo. En particular, la programación en Python y la resolución de problemas de ecuaciones no lineales (métodos de ceros) me costaron mucho trabajo. Tuve que dedicar muchas horas para lograr entender cómo implementar correctamente los algoritmos y cómo lidiar con los errores en los cálculos numéricos.

Además, encontré que la cantidad de contenido fue abrumadora. Se abarcó demasiada materia en poco tiempo, lo que dificultó la comprensión y aplicación profunda de los métodos. Creo que una forma más efectiva de abordar este curso podría haber sido profundizar en uno o dos métodos principales por unidad, en lugar de intentar cubrir una gran cantidad de técnicas. Los métodos restantes podrían haber sido introducidos como temas de interés para los estudiantes que deseen explorarlos por curiosidad, sin que se convirtieran en el foco principal del curso.

El aspecto que más me sorprendió fue la profundidad de la teoría detrás de los métodos numéricos y cómo esto se relaciona con la práctica en la resolución de problemas reales. Los ejercicios de derivadas numéricas y el análisis de métodos para resolver ecuaciones no lineales me ayudaron a entender mejor la precisión de los métodos y las limitaciones que enfrentamos al utilizar aproximaciones numéricas. No obstante, debido a la cantidad de material que se abordó, no siempre pude dedicarle el tiempo necesario a cada tema.

En cuanto a las sugerencias para mejorar el curso, creo que sería muy útil ofrecer más ejemplos prácticos para cada uno de los métodos numéricos, de manera que los estudiantes puedan ver cómo se aplican en contextos específicos antes de abordar los ejercicios más complejos. Además, sería beneficioso dedicar más tiempo a discutir en clase las dificultades que los estudiantes encuentran al implementar los algoritmos, ya que esto podría facilitar una mejor comprensión del material y proporcionar una mayor claridad.

Finalmente, considero que este portafolio ha sido útil, ya que me ha permitido evaluar y reflexionar sobre lo aprendido durante el curso. A pesar de que fue un trabajo que requirió muchas horas, el esfuerzo valió la pena, ya que me permitió mejorar mis habilidades en programación, lo que será de gran utilidad en cursos posteriores y en mi futura carrera profesional en el campo de las ciencias físicas y matemáticas.